Modul 8

Transaktionsverwaltung und Mehrbenutzersynchronisation

Josef Altmann



Der vorliegende Foliensatz basiert auf:

Kemper, A., Leis, V., Eickler, A.: *Datenbanksysteme*, De Gruyter Oldenbourg, 11. Auflage (2026)
Saake, G., Sattler, K.-U., Heuer, A.: *Datenbanken: Implementierungstechniken*, MITP, 3. Auflage (2011)
Weikum, G., Vossen, G.: *Transactional Information Systems*, Morgan Kaufmann Publishers (2002)
Kyte, T., Kuhn, D.: Expert Oracle Database Architecture, Apress, 3rd Edition (2014)
Kleppmann, M.: Datenintensive Anwendungen designen, O'Reilly, 1. Auflage (2019)

In der Realität von Datenbanksystemen kann vieles schiefgehen …

- Datenbanksoftware oder -hardware kann jederzeit ausfallen (auch mitten in einer Schreiboperation).
- Anwendung kann jederzeit abstürzen (auch mitten in einer Folge von Operationen).
- Störungen im Netzwerk können unerwartet die Anwendung von der Datenbank bzw. vom Datenbankknoten abschneiden.
- Mehrere Clients können gleichzeitig in die Datenbank schreiben und dabei ihre Änderungen gegenseitig überschreiben.
- Ein Client kann Daten lesen, die keinen Sinn ergeben, weil sie nur teilweise aktualisiert wurden
- …

WIENER ZEITUNG .at

9. November 2011 | 15:33:46 Uhr

Datenbankproblem führte zu Teilausfall

Bankomat-Netz in Österreich ausgefallen



Ein Datenbankproblem hat am Montag einige Geldautomaten in Österreich lahmgelegt. bilderbox/Erwin Wodicka

Wer am Montagvormittag Geld abheben wollte, der könnte Pech gehabt haben. In ganz Österreich kam es zwischen 10.20 Uhr und 12.59 Uhr zu einem Teilausfall im Bankomet-Netz. Ein technisches Problem habe laut der Betreiberfirma Paylife zu dem Ausfall geführt.

Betroffen von dem Problem waren Bargeldbehebungen. "Seit 12.59 Uhr funktioniert alles wieder klaglos", sagte die Sprecherin der Bankomatfirma Paylife, Angela Szivatz, zur APA. Bei Paylife sprach man von einem Datenbankproblem (Transaktionsproblem), die Geräte selbst waren nicht defekt. An wievielen Stellen der teilweise Systemausfall durchschlug, sagte die Sprecherin nicht.

Betroffen von dem Problem waren Bargeldbehebungen. "Seit 12.59 Uhr funktioniert alles wieder klaglos", sagte die Sprecherin der Bankomatfirma Paylife, Angela Szivatz, zur APA. Bei Paylife sprach man von einem **Datenbankproblem (Transaktionsproblem)**, die Geräte selbst waren nicht defekt. An wievielen Stellen der teilweise Systemausfall durchschlug, sagte die Sprecherin nicht.

Kontoblockade wegen Tankautomatennutzung



Automatentankstellen sind praktisch. Dort gibt es Treibstoff rund um die Uhr und bezahlt wird ganz einfach per Karte. **Dass die Transaktion aber auch gravierende Nachteile mit sich bringen kann**, merken Kunden erst, wenn sie nach dem Tanken ein paar Tage lang nicht mehr voll über ihr Konto verfügen können. Denn um garantiert zu ihrem Geld zu kommen, lassen sich die Unternehmen – noch vor dem Tankvorgang - eine gar nicht so kleine Summe am Kundenkonto reservieren.

http://helpv2.orf.at/stories/1687844/index.html

Überblick

- Motivation
- Transaktionsverwaltung
  - Transaktionskonzept und ACID-Eigenschaften
  - Kommandos einer Transaktionsprache
  - Anforderungen an die Transaktionsverwaltung
- Mehrbenutzersynchronisation
  - Fehler bei Mehrbenutzerbetrieb
  - Serialisierbarkeit
  - Fehlersicherheit
  - Sperrbasierte Verfahren
  - Nicht-sperrbasierte Verfahren
  - Multi-Version Concurrency Control (MVCC)
- Transaktionen in SQL
- Zusammenfassung

© 2026

Motivation



- Beispiel einer typischen Transaktion in einer Bankanwendung:

0. Konsistenter DB-Zustand
1. `read(A, a);` Lese Kontostand von **A** in die Variable **a**
2. `a:= a - 50;` Reduziere den Kontostand um 50 Euro
3. `write(A, a);` Schreibe neuen Kontostand in DB
4. `read(B, b);` Lese den Kontostand von **B** in die Variable **b**
5. `b := b + 50;` Erhöhe Kontostand um 50 Euro
6. `write(B, b);` Schreibe neuen Kontostand in DB
7. Konsistenter DB-Zustand ?

Wozu Tansaktionen?

- Beispielszenarien
  - Überschneidende **Kontooperationen** einer Bank
    - Mehrfache Abhebungen von einem Konto von unterschiedlichen Bankomaten aus können zu nicht erlaubten Überziehungen führen
      - Gefahr des Geldverlusts
  - Gleichzeitige **Platzreservierung** aus Reisebüros bzw. auf Buchungsplattformen
    - Platz könnte mehrfach verkauft werden, wenn mehrere Reisebüros den Platz als verfügbar identifizieren
      - Gefahr der Überbuchung
  - **Statistische Analysen** und Datenbankoperationen
    - Ergebnisse sind verfälscht, wenn während der Berechnung Daten geändert werden
  - ...

➢ Transaktion als Ausführungseinheit stellt **Integrität** sicher!

Was ist eine Transaktion?

- Eine **Transaktion** als Verarbeitungseinheit ist
  - eine Folge von Lese- und Änderungsoperationen
  - mit einem definierten Beginn und einem definierten Abschluss
  - die die DB von einem konsistenten Zustand in einen konsistenten, eventuell veränderten, Zustand überführt

- Integritätsaspekte:
  - **Semantische Integrität**
    - Korrekter (konsistenter) DB-Zustand bei Transaktionsende
  - **Ablaufintegrität**
    - Fehler durch „gleichzeitigen" Zugriff mehrerer Benutzer auf dieselben Daten vermeiden

➢ **Transaktionsverwaltung** ist Kernaufgabe eines DBMS!

Was stellt eine Transaktion sicher?

- Datenbanksystem garantiert für eine Transaktion folgende Eigenschaften:

A	**A**tomicity (Atomarität oder Unteilbarkeit)
C	**C**onsistency (Konsistenz oder Integritätserhaltung)
I	**I**solation (Mehrbenutzerkontrolle)
D	**D**urability (Dauerhaftigkeit oder Persistenz)

➢ Diese Eigenschaften werden als **ACID-Regel** bezeichnet.

Was sind ACID-Eigenschaften?

- **A**tomicity: Atomarität oder Unteilbarkeit
  - Transaktion wird entweder ganz oder gar nicht ausgeführt
  - Nach Abbruch keine Zwischenergebnisse in der DB gespeichert
- **C**onsistency: Konsistenz oder Integritätserhaltung
  - DB ist vor Beginn und nach Beendigung einer Transaktion jeweils in einem konsistenten Zustand
  - Nach Abbruch muss immer ein konsistenter DB-Zustand hergestellt werden
- **I**solation: Mehrbenutzerbetrieb
  - Nutzer, der mit einer DB arbeitet, sollte den Eindruck haben, dass er mit dieser DB alleine arbeitet
- **D**urability: Dauerhaftigkeit oder Persistenz
  - Nach erfolgreichem Transaktionsabschluss muss das Ergebnis dieser Transaktion „dauerhaft“ in der DB gespeichert werden

© 2026

Kommandos einer Transaktionssprache

- `begin of transaction (bot)`
  - kennzeichnet den Beginn einer Transaktion (in SQL implizit!)
- `commit`
  - alle Änderungen der Transaktion in der DB dauerhaft festschreiben
- `abort (rollback)`
  - bricht Transaktion ab (explizit oder implizit)
  - DB muss sicherstellen, dass Datenbasis wieder in den Zustand zurückversetzt wird, der vor der Transaktion existierte
- `define savepoint`
  - definiert einen Sicherungspunkt, auf den sich die noch aktive Transaktion zurücksetzen lässt
  - Änderungen bis zu diesem Sicherungspunkt werden **noch nicht** in die Datenbasis eingebracht (es kann noch ein `abort` geben)
- `backup transaction`
  - aktive Transaktion wird auf den jüngsten (letzten) Sicherungspunkt zurückgesetzt

© 2026

Arbeiten mit Transaktionen

- Transaktionslose DB-Operationen
  - nicht erlaubt!
  - jeder lesende oder schreibende Zugriff auf die DB kann nur innerhalb einer Transaktion stattfinden
- Transaktions-Start
  - explizit mit einem `begin of transaction`-Befehl oder implizit mit dem ersten SQL-Befehl
- Korrekter Transaktions-Abschluss
  - nur mit dem `commit`-Befehl korrekt abgeschlossen
  - andernfalls nicht korrekt beendet
- Transaktions-Abbruch
  - explizit mit `rollback` oder implizit durch ein äußeres Ereignis (Fehler, Constraint-Verletzung, Verbindungsabbruch, …)

Anforderungen an Transaktionen

1. Wiederherstellung und Fehlerbehebung (**Recovery**)
   - Datenbank gegen Soft- und Hardware-Fehler schützen
   - ➢ gewährleistet **A**tomarität und **D**auerhaftigkeit

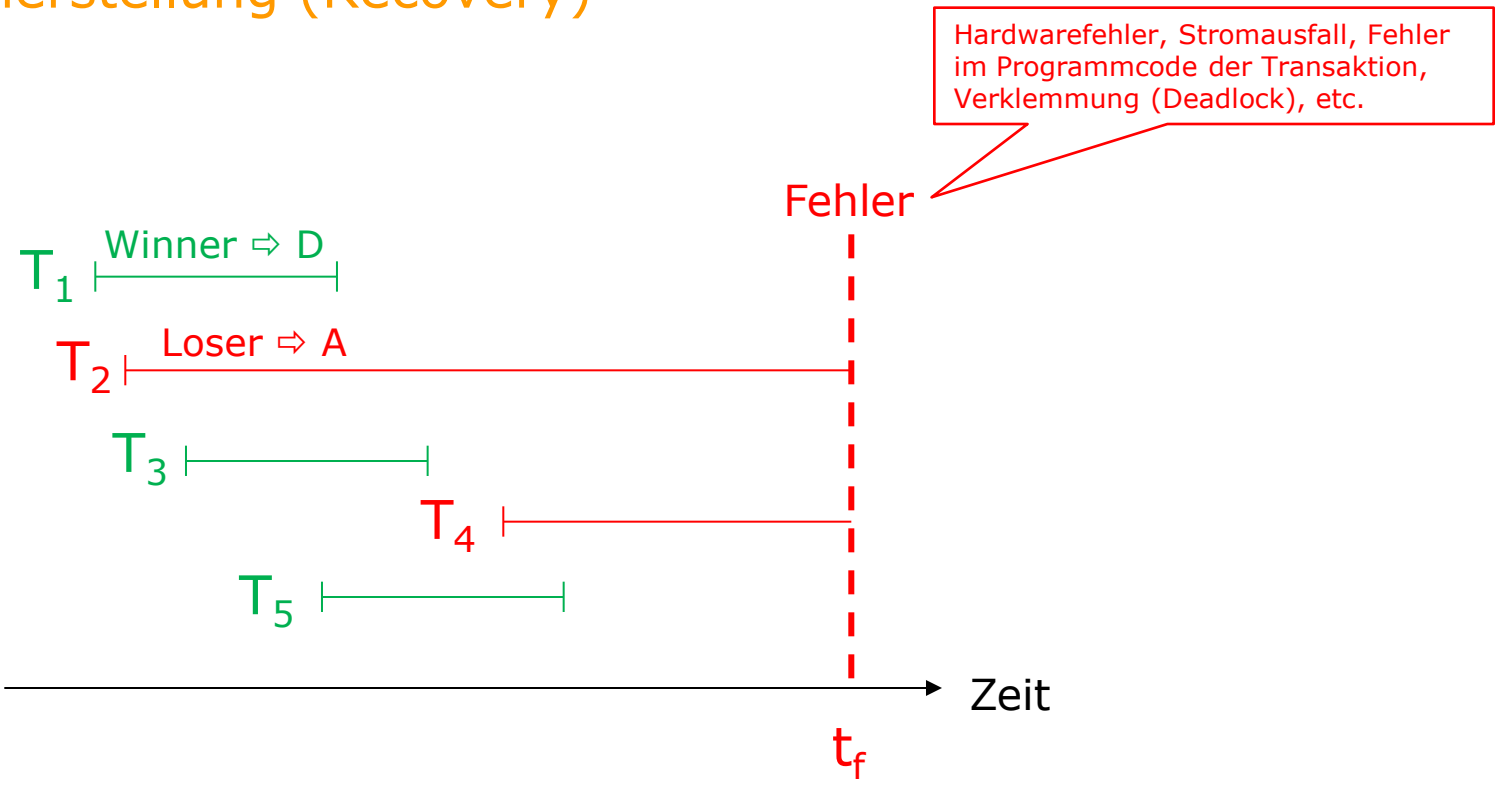
2. Integritätsprüfung und -kontrolle
   - Integritätsbedingungen gemäß Schema müssen erfüllt sein
   - ➢ gewährleistet einen konsistenten (**C**onsistency) Datenbankzustand bei Transaktionsende

3. Mehrbenutzersynchronisation (**Concurrency control**)
   - gleichzeitig ablaufende Transaktionen synchronisieren
   - ➢ gewährleistet die **I**solation von parallel ablaufenden Transaktionen

© 2026

1 Transaktion + Fehlerbehandlung (A+D)

Wiederherstellung (Recovery)



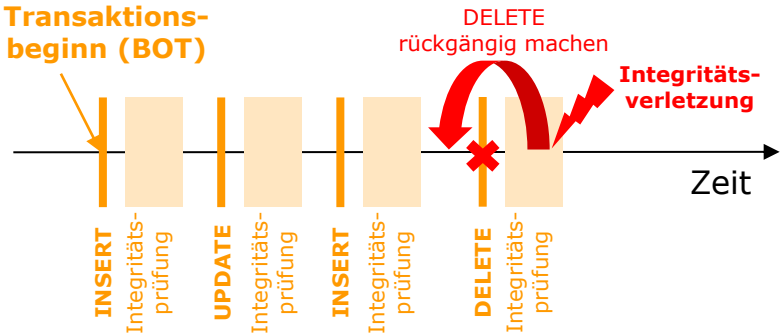
- Folgen:
  - Inhalt des Speichers zum Zeitpunkt t_f ist unbrauchbar!
  - ➢ Transaktionen in unterschiedlicher Weise davon betroffen
- Transaktionszustände:
  - zum Fehlerzeitpunkt noch aktive Transaktionen
    - T_2 und T_4
  - bereits vor dem Fehlerzeitpunkt beendete Transaktionen
    - T_1, T_3 und T_5

© 2026

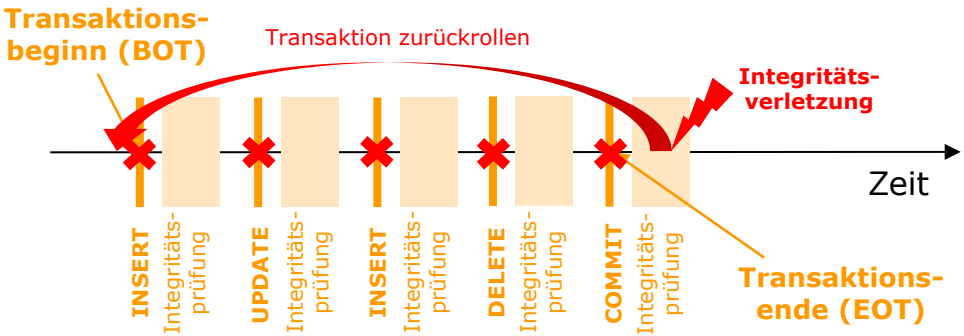
2 Transaktion + Datenkonsistenz (C)

Integritätskontrolle

- Beispiel:
  - Übertragung eines Betrages X von Giro-Konto G1 auf Sparbuch-Konto S1
  - Integritätsbedingung: Summe der Kontostände bleibt konstant
- Realisierung als Sequenz zweier elementarer Änderungen
  - Abhängig vom Prüfungszeitpunkt (spätestens aber zu Transaktionsende) müssen alle Integritätsbedingungen erfüllt sein!



Unmittelbare Integritätsprüfung (**IMMEDIATE**)

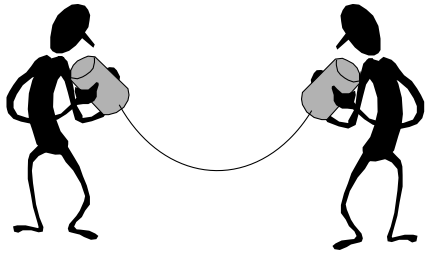


Verzögerte Integritätsprüfung (**DEFERRED**)

3 Transaktion + Mehrbenutzerbetrieb (I)

Mehrbenutzerkontrolle (Concurrency control)

- Beispiel:
  - Zwei Anwender bearbeiten die gleiche Tabelle gleichzeitig



Anwender 1

```
SELECT x FROM Tab_A;

y := x + 5;

UPDATE Tab_A SET x := y;
```

Anwender 2

```
SELECT x FROM Tab_A;

y := x + 10;

UPDATE Tab_A SET x := y;
```

- Was passiert, wenn beide Anwender gleichzeitig auf den selben Datenbestand zugreifen?

Überblick

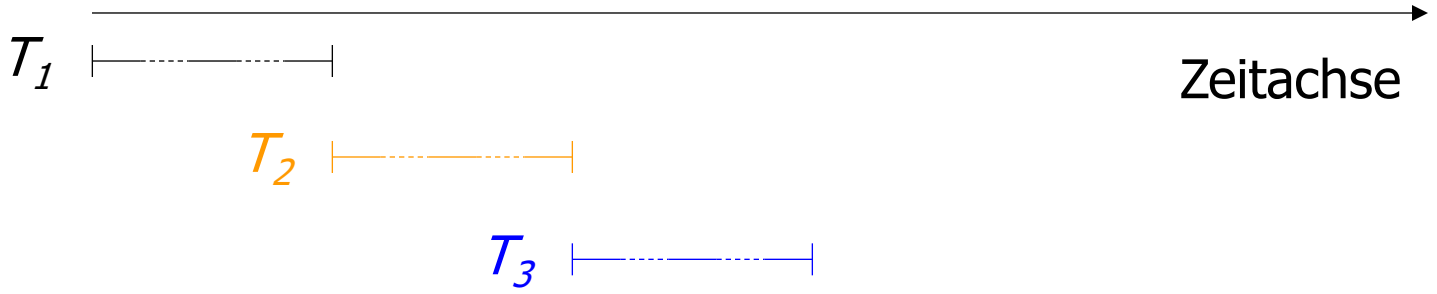
- Motivation
- Transaktionsverwaltung
- Mehrbenutzersynchronisation
  - Fehler bei Mehrbenutzerbetrieb
    - Verlorengegangene Änderungen (lost update)
    - Lesen von nicht freigegebenen Änderungen (dirty read)
    - Nicht wiederholbares Lesen (non-repeatable Read)
    - Lesen inkonsistenter Zustände
    - Phantom-Problem
    - Cursor-Referenzen
  - Serialisierbarkeit
  - Fehlersicherheit
  - Sperrbasierte Verfahren
  - Nicht-sperrbasierte Verfahren
  - Multi-Version Concurrency Control (MVCC)
- Transaktionen in SQL
- Zusammenfassung

© 2026

Mehrbenutzersynchronisation

- Ausführung der Transaktionen T_1, T_2 und T_3:

a) im Einzelbetrieb und



b) im (verzahnten) Mehrbenutzerbetrieb



Fehler bei Mehrbenutzerbetrieb

Verlorengegangene Änderungen (lost update)



Benutzer A



Benutzer B

Zeit	T_1	Wert		Wert	T_2
1.	SELECT Wert FROM Tabelle	5		5	SELECT Wert FROM Tabelle
2.					
3.	Puffer = Wert + 1	5		5	Puffer = Wert + 2
4.					
5.	UPDATE Tabelle SET Wert=Puffer	6		7	UPDATE Tabelle SET Wert=Puffer
6.					
7.	SELECT Wert FROM Tabelle	7		7	SELECT Wert FROM Tabelle
8.					

Datenbank

Aktualisierung von Benutzer A geht verloren!

Fehler bei Mehrbenutzerbetrieb

Lesen von nicht freigegebenen Änderungen (dirty read)

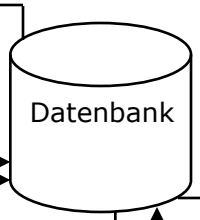


Benutzer A



Benutzer B

Zeit	T_1	Wert	Wert	T_2
1.	SELECT Wert FROM Tabelle	5		
2.				
3.	Puffer = Wert + 1	5		
4.				
5.	UPDATE Tabelle SET Wert=Puffer	6		
6.			6	SELECT Wert FROM Tabelle
7.				
8.			6	Puffer = Wert + 1
9.				
10.			7	UPDATE Tabelle SET Wert=Puffer
11.	abort (->Rollback Wert=5)	5		
12.			5	SELECT Wert FROM Tabelle



Benutzer B hat ungültigen Wert gelesen!

Fehler bei Mehrbenutzerbetrieb

Nicht wiederholbares Lesen (non-repeatable read)



Benutzer A



Benutzer B

Zeit	T_1	Wert	Wert	T_2
1.	`SELECT Wert FROM Tabelle`	5		
2.	`Puffer = Wert`			
3.			5	`SELECT Wert FROM Tabelle`
4.				`Puffer = Wert + 1`
5.			6	`UPDATE Tabelle SET Wert=Puffer`
6.	`SELECT *`	**not**		
7.	`FROM Tabelle`	**found**		
8.	`WHERE Wert = Puffer`			

Datenbank

Erstes Leseergebnis von `Wert` ist nicht wiederholbar!

Fehler bei Mehrbenutzerbetrieb

Lesen inkonsistenter Zustände

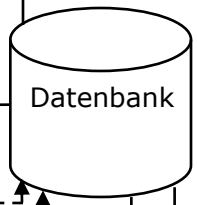


Benutzer A



Benutzer B

Zeit		T_1	Wert X	Wert Y	T_2
	1.	`SELECT X FROM Tabelle`	5		
	2.	`X_Puffer = X - 1`			
	3.				
	4.	`SELECT Y FROM Tabelle`		5	
	5.	`Y_Puffer = Y + 1`			
	6.				
	7.	`UPDATE Tabelle SET X = X_Puffer`	4		
	8.				
	9.				
	10.				`SELECT X FROM Tabelle`
	11.	`UPDATE Tabelle SET Y = Y_Puffer`		6	`SELECT Y FROM Tabelle`
	12.	`COMMIT`			



4

5

erfüllt — Integritätsbedingung: `X + Y = 10` — nicht erfüllt

(Variante von Non-Repeatable Read)

Fehler bei Mehrbenutzerbetrieb

Phantomproblem



Benutzer A



Benutzer B

Zeit

Schritt	T_1	T_2
1.	`SELECT COUNT (*)`	
2.	`INTO` Anzahl	
3.	`FROM` Mitarbeiter;	
4.		`INSERT`
5.		`INTO` Mitarbeiter
6.		`VALUES` (Meier, 500,…);
7.		`COMMIT;`
8.	`UPDATE` Mitarbeiter	
9.	`SET` Gehalt = Gehalt +	
10.	10000/Anzahl;	
11.	`COMMIT;`	

Fehler bei Mehrbenutzerbetrieb

Cursor-Referenzen



Benutzer A



Benutzer B

Zeit

Schritt	T_1	T_2
1.	`DECLARE mit_cursor CURSOR FOR`	
2.	`SELECT *`	
3.	`FROM Mitarbeiter`	
4.	`WHERE Abt = 'Verkauf';`	
5.	`...`	
6.		`UPDATE Mitarbeiter`
7.		`SET Abt = 'Einkauf'`
8.		`WHERE MName = ...;`
9.		`...`
10.	`FETCH mit_cursor INTO ...`	

Überblick

- Motivation
- Transaktionsverwaltung
- Mehrbenutzersynchronisation
  - Fehler bei Mehrbenutzerbetrieb
  - Serialisierbarkeit
    - Das Read/Write-Modell
    - Der Begriff des „Schedules“
    - Serielle und Verzahnte Schedules
    - Effektäquivalenz
    - Konfliktmatrix
    - Konfliktserialisierbarkeit (CSR)
    - Graph-basierter Test auf CSR

    } Theoretische Grundlage!
  - Fehlersicherheit
  - Sperrbasierte Verfahren
  - Nicht-sperrbasierte Verfahren
  - Multi-Version Concurrency Control (MVCC)
- Transaktionen in SQL
- Zusammenfassung

© 2026

Woraus besteht eine Transaktion?

Das Read/Write-Modell

- Zur Vereinfachung der Betrachtungen
  - **Abstraktion** von **konkreten DB-Operationen** und **DB-Objekten** durch das sog. **Read/Write-Modell**
- Eine **Transaktion** (T_i)
  - … ist eine Folge von **DB-Operationen** (o_j)
  - … die **DB-Objekte** (x)
  - … **lesen** $r_i(x)$ oder
  - … **schreiben** $w_i(x)$.
  - … wird beendet durch
    - Abbruch (a_i) oder
    - Commit (c_i)
  - … und somit zu einer vollständigen Transaktion (T_i)

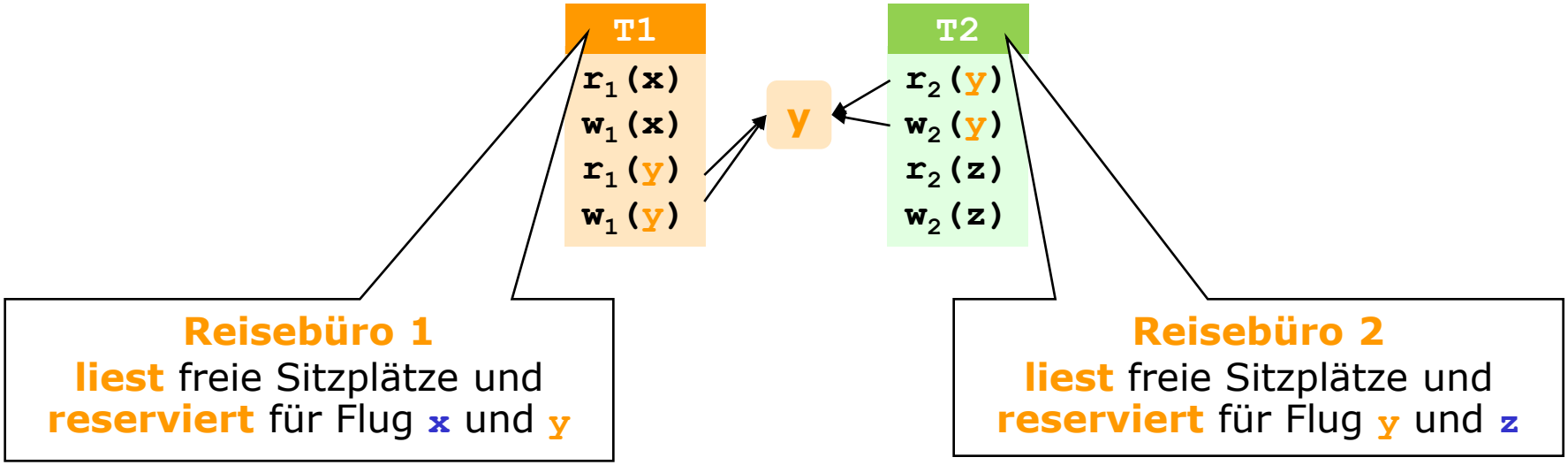
Ausführungsplan (Schedule)

$T_i = o_1\ o_2\ o_3\ \ldots\ o_n$
mit $o_j \in \{r_i(x);\ w_i(x)\}$

- Eine **einzelne Transaktion** T_i erzeugt gemäß Consistency-Eigenschaft **immer korrekte Effekte**

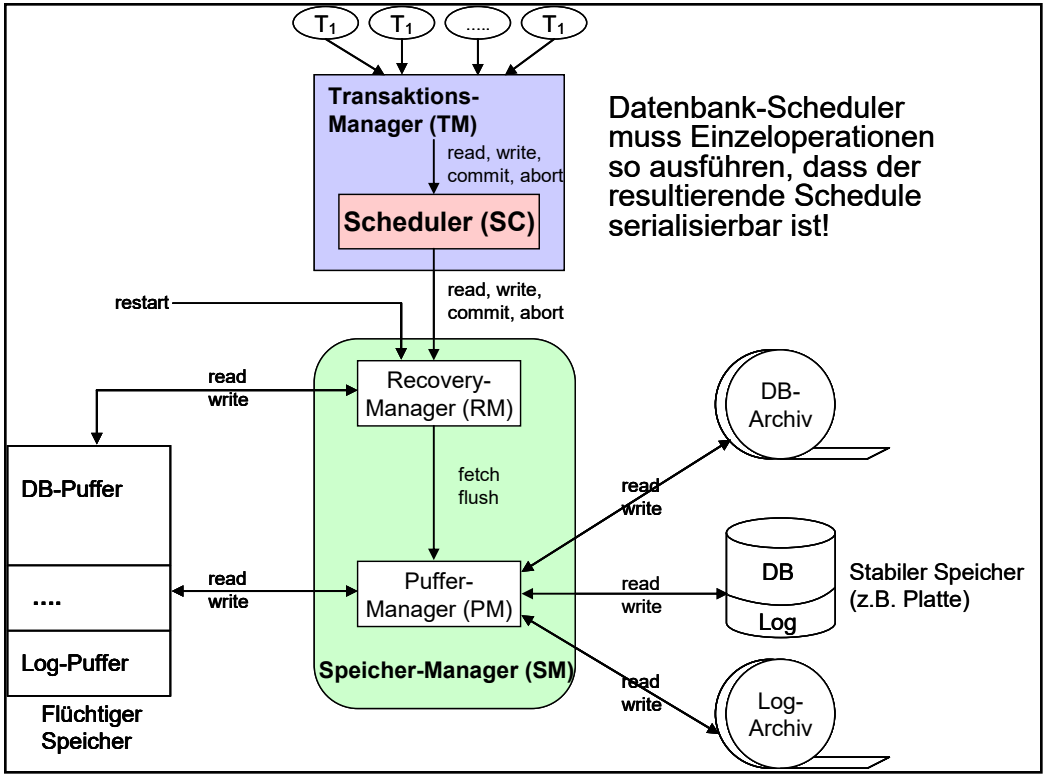
Wann sind Effekte mehrerer Transaktionen korrekt?
Serialisierbarkeit

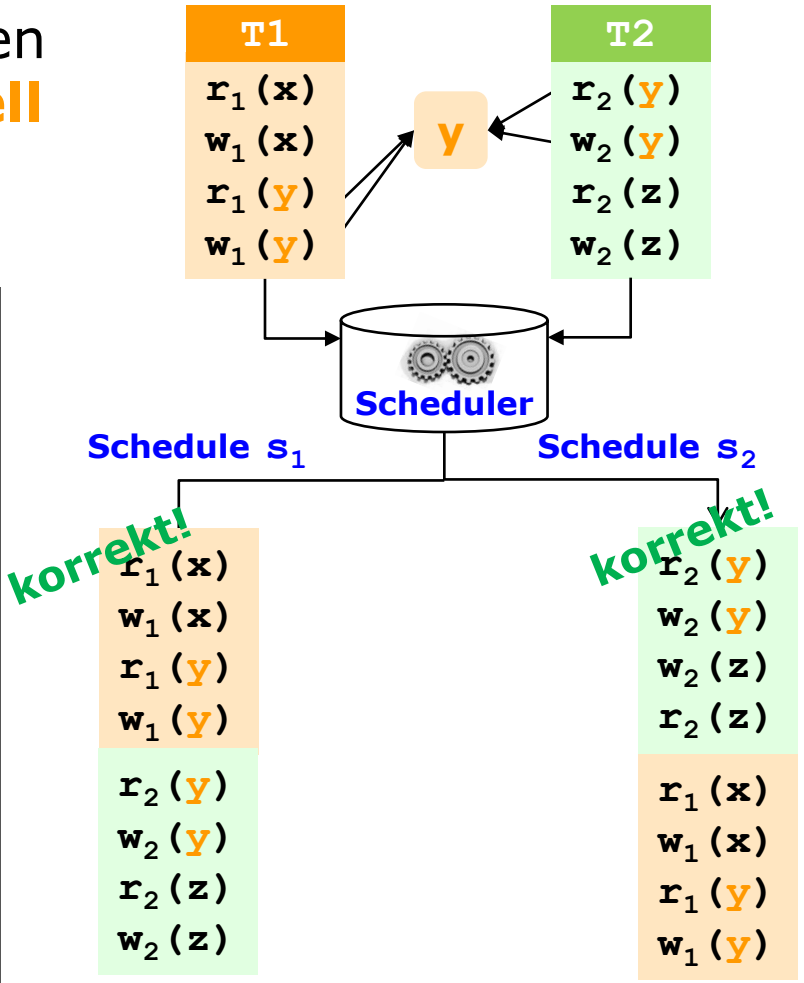
- **Mehrere** Transaktionen erzeugen **korrekte Effekte** falls sie **seriell** ausgeführt werden
  - ... da vollständige Isolation gegeben
  - ... jedoch keine Parallelität (verzahnte Ausführung) möglich



Wann sind Effekte mehrerer Transaktionen korrekt?
Serialisierbarkeit

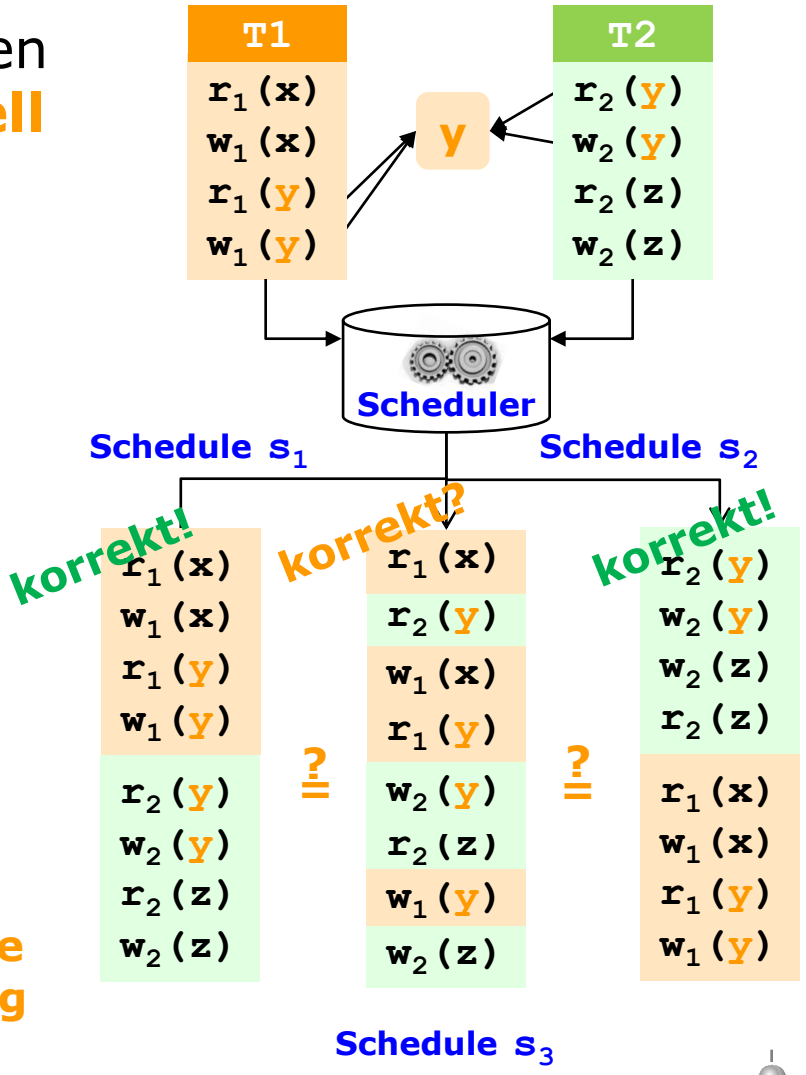
- **Mehrere** Transaktionen erzeugen **korrekte Effekte** falls sie **seriell** ausgeführt werden
  - … da vollständige Isolation gegeben





Wann sind Effekte mehrerer Transaktionen korrekt?
Serialisierbarkeit

- **Mehrere** Transaktionen erzeugen **korrekte Effekte** falls sie **seriell** ausgeführt werden
  - … da vollständige Isolation gegeben
- **ABER**
  - **Verzahnte Ausführung** aus **Performanz-Gründen** notwendig
  - Die **lokale Reihenfolge** der Einzelschritte jeder Transaktion wird im **Schedule** beibehalten
- **Korrekt falls:**
  - Verzahnte Ausführung **serialisierbar** ist
  - d.h. **äquivalente Effekte** liefert **wie** eine **beliebige serielle Ausführung**



Wann sind Effekte mehrerer Transaktionen korrekt?

Serialisierbarkeit

```
T1: read A; A:=A-10; write A; read B; B:=B+10; write B;
T2: read B; B:=B-20; write B; read C; C:=C+20; write C;
```

Zeit

Schedule 1		Schedule 2		Schedule 3	
T_1	T_2	T_1	T_2	T_1	T_2
read A		read A		read A	
A – 10			read B	A – 10	
write A		A – 10			read B
read B			B – 20	write A	
B + 10		write A			B – 20
write B			write B	read B	
	read B	read B			write B
	B – 20		read C	B + 10	
	write B	B + 10			read C
	read C		C + 20	write B	
	C + 20	write B			C + 20
	write C		write C		write C
Serieller Schedule		Verzahnte/parallele Schedules			

Effekte der Schedules

	A	B	C	A+B+C
initialer Wert	10	10	10	30
nach Ausführung 1	0	0	30	30
nach Ausführung 2	0	0	30	30
nach Ausführung 3	**0**	**20**	**30**	**50**

Wann ist Effekt-Äquivalenz gegeben?

Reihenfolgeabhängigkeit - Konfliktserialisierbarkeit

- Der **Effekt eines Schedules** ist **abhängig** von der **Reihenfolge** in der **"konfliktäre" Operationen** ausgeführt werden
- **Zwei Operationen** o_1, o_2 sind konfliktär wenn
  - sie zu **verschiedenen Transaktionen** (T_1, T_2) gehören und
  - sie auf das **gleiche DB-Objekt** (y) zugreifen und
  - mindestens eine Operation eine **Schreib-Operationen** ist

Konfliktmatrix

T_1 \ T_2	$r_2(y)$	$w_2(y)$
$r_1(y)$	✓	✗
$w_1(y)$	✗	✗

Verschiedene Schedules – Effekt ist ...

$r_1(y)$ $r_2(y)$	=	$r_2(y)$ $r_1(y)$	**... unabhängig** von der Reihenfolge
$w_1(y)$ $r_2(y)$	≠	$r_2(y)$ $w_1(y)$	**... abhängig** von der Reihenfolge
$r_1(y)$ $w_2(y)$	≠	$w_2(y)$ $r_1(y)$	**... abhängig** von der Reihenfolge
$w_1(y)$ $w_2(y)$	≠	$w_2(y)$ $w_1(y)$	**... abhängig** von der Reihenfolge

Wann ist Effekt-Äquivalenz gegeben?

Reihenfolgeabhängigkeit - Konfliktserialisierbarkeit

- Der **Effekt eines Schedules** ist **abhängig** von der **Reihenfolge** in der **"konfliktäre" Operationen** ausgeführt werden
- **Zwei Operationen** ... sind konfliktär wenn

Schlussfolgerung

→ Falls die **Reihenfolge** aller **konfliktären Operationen** im **verzahnten** Schedule **gleich** ist jener eines **seriellen** Schedules

→ Dann ist **Effekt-Äquivalenz im Sinne der Konfliktserialisierbarkeit** gegeben

→ Als "**CSR**" wird die Menge aller konfliktserialisierbaren Schedules bezeichnet

→ **Wie** kann man Konfliktserialisierbarkeit **überprüfen**?

$w_1(y)$ $w_2(y)$ ≠ $w_2(y)$ $w_1(y)$ ... abhängig von der Reihenfolge

Banktransaktionen:

- T_1 transferiere einen bestimmten Betrag von A nach B
- T_2 transferiere einen bestimmten Betrag von C nach A

T_1	T_2
$r(A, a_1)$	
	$r(C, c_2)$
$a_1 := a_1 - 50$	
$w(A, a_1)$	
	$c_2 := c_2 + 100$
	$w(C, c_2)$
$r(B, b_1)$	
$b_1 := b_1 + 50$	
$w(B, b_1)$	
COMMIT	
	$r(A, a_2)$
	$a_2 := a_2 - 100$
	$w(A, a_2)$
	COMMIT

⇨ *1

T_1	T_2
$r_1(A)$	
	$r_2(C)$
$w_1(A)$	
	$w_2(C)$
$r_1(B)$	
$w_1(B)$	
c_1	
	$r_2(A)$
	$w_2(A)$
	c_2

⇩ *2

*1 Aus der Sicht der Mehrbenutzersynchronisation sind nur die elementaren Datenbankoperationen read, write, insert und delete relevant, da die Bearbeitung lokaler Variablen nicht von der Nebenläufigkeit betroffen ist.

*2 $S = r_1(A), r_2(C), w_1(A), w_2(C), r_1(B), w_1(B), c_1, r_2(A), w_2(A), c_2$

Äquivalenz zweier Historien (Schedules)

(bzw. Hinführung zur Konflikt-Serialisierbarkeit)

Beispiel Überweisungstransaktion

① $r_1(A)$; $r_2(C)$; $w_1(A)$; $w_2(C)$; $r_1(B)$; $w_1(B)$; c_1; $r_2(A)$; $w_2(A)$; c_2

$r_2(C)$ und $w_1(A)$ sind zueinander nicht konfliktär und können vertauscht werden (irrelevant bzgl. Äquivalenz)

② $r_1(A)$; $w_1(A)$; $r_2(C)$; $w_2(C)$; $r_1(B)$; $w_1(B)$; c_1; $r_2(A)$; $w_2(A)$; c_2

$r_1(B)$ steht mit $w_2(C)$ ~~und~~ und $r_2(C)$ nicht in Konflikt

→ zweifache

③ $r_1(A)$; $w_1(A)$; $r_1(B)$; $r_2(C)$; $w_2(C)$; $w_1(B)$; c_1; $r_2(A)$; $w_2(A)$; c_2

$w_1(B)$ kann an $w_2(C)$ und $r_2(C)$ vorbeigetauscht werden, da kein Konflikt besteht. c_1 kann man ebenfalls nach vorne bringen.

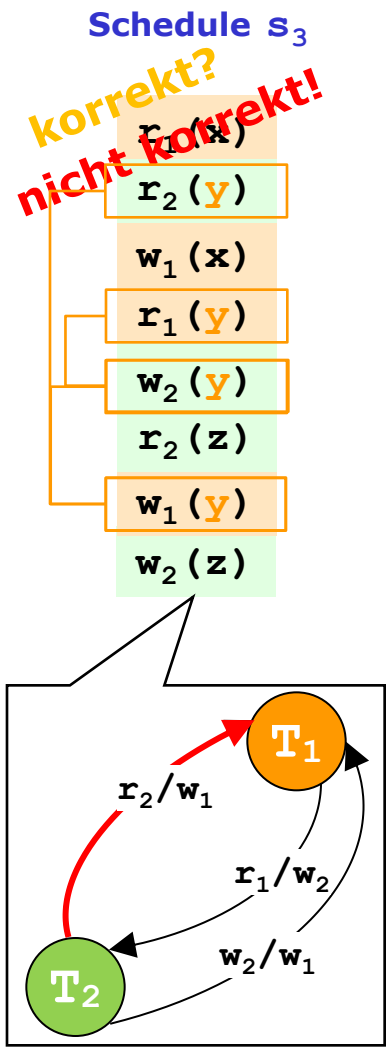
$r_1(A)$, $w_1(A)$, $r_1(B)$, $w_1(B)$, c_1, $r_2(C)$, $w_2(C)$, $r_2(A)$, $w_2(A)$, c_2

→ durch sukzessives, zielgerichtetes Vertauschen von Operationen, die nicht in Konflikt stehen, ist aus dem ursprünglichen Schedule ein serieller Schedule geworden!! → beide sind äquivalent!

(= topologisches Sortieren)

Wie ist Konfliktserialisierbarkeit überprüfbar?

Konfliktgraph



Konfliktgraph $G(s)=(N,E)$ von Schedule s

- **Knoten**menge N enthält alle **Transaktionen** von s
- **Kanten**menge E enthält alle **konfliktären Operationen** als gerichtete Kanten

Konfliktmatrix ...

T_1 \ T_2	$r_2(y)$	$w_2(y)$
$r_1(y)$	✓	✗
$w_1(y)$	✗	✗

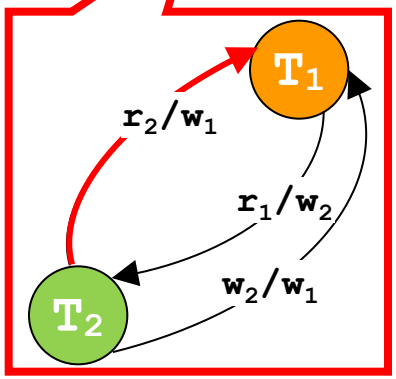
Wie ist Konfliktserialisierbarkeit überprüfbar?

Konfliktgraph

Schedule s_3

nicht korrekt!

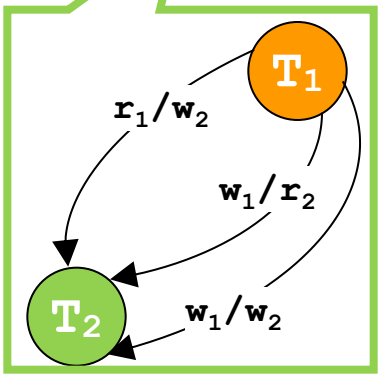
$r_1(x)$ $r_2(y)$ $w_1(x)$ $r_1(y)$ $w_2(y)$ $r_2(z)$ $w_1(y)$ $w_2(z)$



Schedule s_1

korrekt!

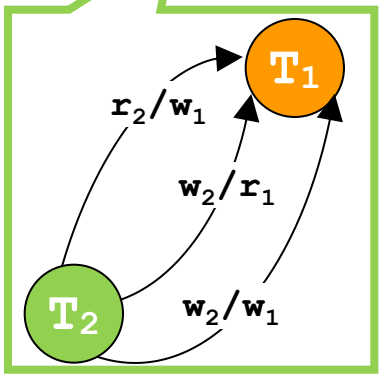
$r_1(x)$ $w_1(x)$ $r_1(y)$ $w_1(y)$ $r_2(z)$ $w_2(z)$ $r_2(y)$ $w_2(y)$



Schedule s_2

korrekt!

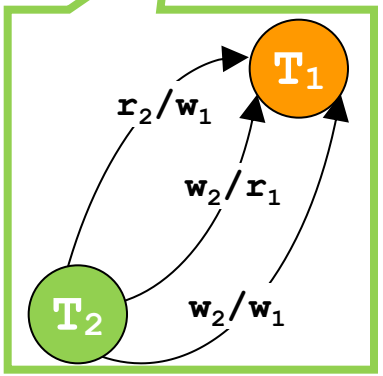
$r_2(y)$ $w_2(y)$ $r_2(z)$ $w_2(z)$ $r_1(x)$ $w_1(x)$ $r_1(y)$ $w_1(y)$



Schedule s_4

korrekt!

$r_1(x)$ $r_2(y)$ $w_1(x)$ $w_2(y)$ $r_1(y)$ $r_2(z)$ $w_1(y)$ $w_2(z)$



Wie ist Konfliktserialisierbarkeit überprüfbar?

Konfliktgraph

Schedule s_3 — nicht korrekt!

$r_1(x)$
$r_2(y)$

Schedule s_1 — korrekt!

$r_1(x)$
$w_1(x)$
$r_1(y)$

Schedule s_2 — korrekt!

$r_2(y)$
$w_2(y)$

Schedule s_4 — korrekt?

$r_1(x)$
$r_2(y)$

Zusammenfassung

→ Falls Konfliktgraph **azyklisch** … **korrekter Effekt**

- **Serielle** Schedules **immer**
- **Verzahnte Schedules** mit **azyklischen** Konfliktgraphen sind **in serielle überführbar** (z.B. durch topologisches Sortieren)

→ Falls Konfliktgraph **zyklisch** … **inkorrekter Effekt – nicht konfliktserialisierbar**

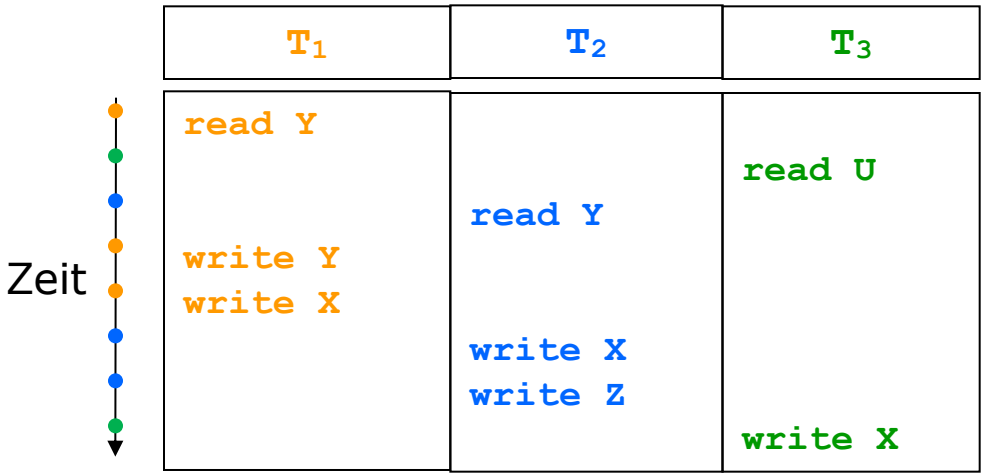
Wie ist Konfliktserialisierbarkeit überprüfbar?

Konfliktgraph – Regeln für Aufbau

- Erzeuge für jede Transaktion T_i, die in **s** auftritt,
  - einen Knoten T_i im Serialisierbarkeitsgraphen (SG)
- Erzeuge für jeden Fall in **s**, bei dem T_j ein **read(x)** ausführt, nachdem T_i ein **write(x)** ausgeführt hat,
  - eine Kante ($T_i \rightarrow T_j$) im SG.
- Erzeuge für jeden Fall in **s**, bei dem T_j ein **write(x)** ausführt, nachdem T_i ein **read(x)** ausgeführt hat,
  - eine Kante ($T_i \rightarrow T_j$) im SG.
- Erzeuge für jeden Fall in **s**, bei dem T_j ein **write(x)** ausführt, nachdem T_i ein **write(x)** ausgeführt hat,
  - eine Kante ($T_i \rightarrow T_j$) im SG.
- Der Ausführungsplan **s** ist **serialisierbar**,
  - falls der SG **keinen Zyklus** aufweist.

Wie ist Konfliktserialisierbarkeit überprüfbar?

Konfliktgraph – Weiteres Beispiel



Zugehöriger Schedule:

s = $r_1(y)\, r_3(u)\, r_2(y)\, w_1(y)\, w_1(x)\, w_2(x)\, w_2(z)\, w_3(x)$

Wie ist Konfliktserialisierbarkeit überprüfbar?

Konfliktgraph – Weiteres Beispiel

Schedule (Ausführungsplan):

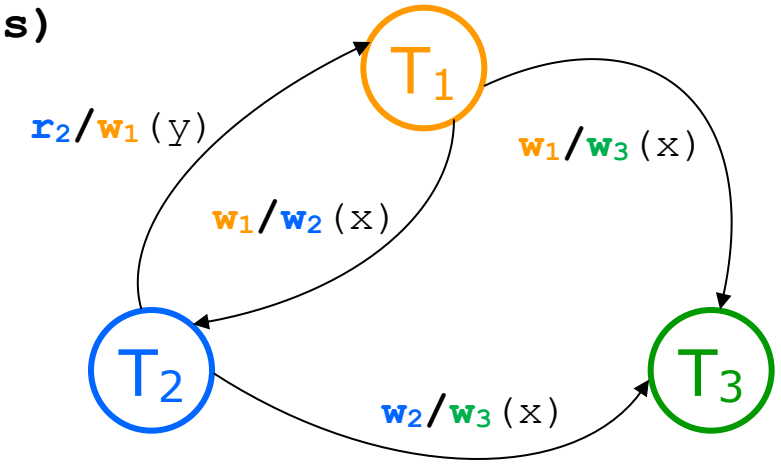
$s = r_1(y)\, r_3(u)\, r_2(y)\, w_1(y)\, w_1(x)\, w_2(x)\, w_2(z)\, w_3(x)$

Konfliktrelation:

$C(s) = \{ (r_2(y), w_1(y)), (w_1(x), w_2(x)), (w_1(x), w_3(x)), (w_2(x), w_3(x)) \}$

Konfliktgraph:

$G(s)$



Keine äquivalente serielle Schedules

Grund: Zyklus

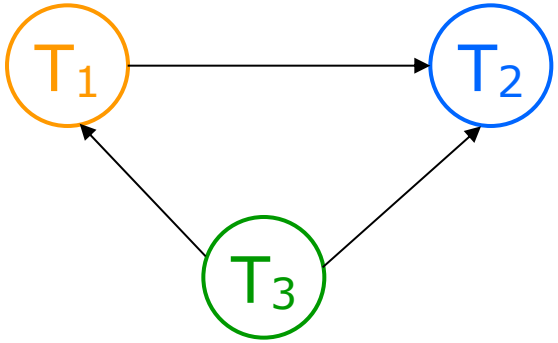
$x(T_1 \rightarrow T_2),\ y(T_2 \rightarrow T_1)$

Hinweis: Ein Zyklus in einem Konfliktgraphen muss sich nicht auf gleiche Datenelemente beziehen, die beteiligten Kanten können sich auch auf verschiedene Datenelemente beziehen.

Wie ist Konfliktserialisierbarkeit überprüfbar?

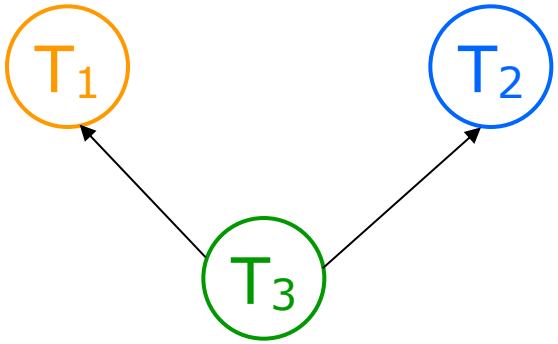
Konfliktgraph – Weiteres Beispiel

Beispiele für äquivalente serielle Schedules für folgende Konfliktgraphen `G(s)`:



Äquivalenter serieller Schedule

(1) $T_3 \rightarrow T_1 \rightarrow T_2$



Äquivalente serielle Schedules

(1) $T_3 \rightarrow T_1 \rightarrow T_2$

(2) $T_3 \rightarrow T_2 \rightarrow T_1$

Überblick

- Motivation
- Transaktionsverwaltung
- Mehrbenutzersynchronisation
  - Fehler bei Mehrbenutzerbetrieb
  - Serialisierbarkeit
  - Fehlersicherheit
    - Rücksetzbarkeit (RC)
    - Vermeidung Kaskadierender Abbrüche (ACA)
    - Striktheit (ST)
    - Serialisierbarkeit und Fehlersicherheit
  - Sperrbasierte Verfahren
  - Nicht-sperrbasierte Verfahren
  - Multi-Version Concurrency Control (MVCC)
- Transaktionen in SQL
- Zusammenfassung

© 2026

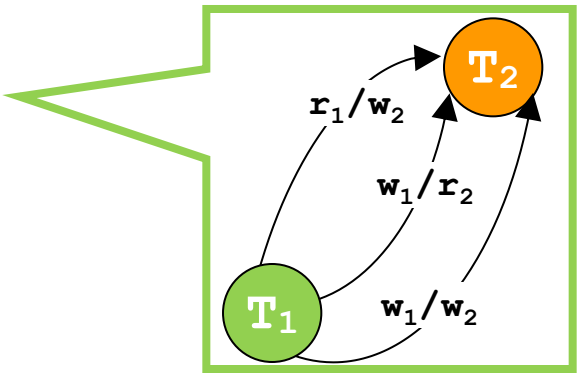
Fehlersicherheit

- Bisher wurde der **Fehlerfall ausgeschlossen**!
- Schedule soll so gestaltet sein, dass **jede Transaktion** zu jedem Zeitpunkt vor Ausführung eines `COMMIT` **zurückgesetzt** werden kann, ohne dass davon andere Transaktionen beeinträchtigt werden.
- Aus Sicht der Fehlersicherheit ist folgender Schedule `s` **nicht akzeptabel**:

Dirty Read

$s = r_1(x)\ w_1(x)\ r_2(x)\ a_1\ w_2(x)\ c_2$

... aber serialisierbar in CSR!



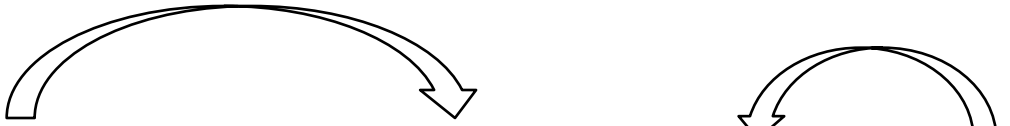
- ➢ zusätzliche Kriterien notwendig, um Fehlersituation zu vermeiden!

Rücksetzbarkeit – RC

- Schedule s heißt **rücksetzbar** (recoverable), falls folgende Bedingung erfüllt ist:
  - $\forall_{i \neq j, v}\ \mathrm{w}_j(v) \rightarrow_s \mathrm{r}_i(v) \wedge \mathrm{c}_i \in s \Rightarrow \mathrm{c}_j \rightarrow_s \mathrm{c}_i$
  - T_i liest Wert v den T_j geschrieben hat
  - Commit c_i ist Teil des Schedule s
  - dann muss
    - Commit von T_j (c_j) vor Commit von T_i (c_i) kommen
- Anders ausgedrückt:
  - Eine Transaktion darf erst dann ihr `COMMIT` durchführen, wenn alle Transaktionen, von denen sie gelesen hat, beendet sind.

Rücksetzbarkeit – RC

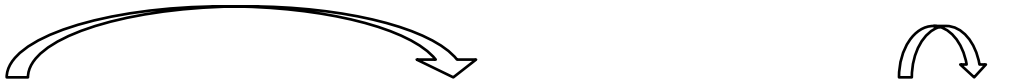
Beispiel



- $s_1 = w_1(x)\, w_1(y)\, r_2(u)\, w_2(x)\, r_2(y)\, w_2(y)\, c_2\, w_1(z)\, c_1$

In s_1 liest T_2 das Datenobjekt y von T_1, aber c_2 kommt vor c_1

⇨ s_1 ist gemäß ACID-Eigenschaft nicht rücksetzbar



- $s_2 = w_1(x)\, w_1(y)\, r_2(u)\, w_2(x)\, r_2(y)\, w_2(y)\, w_1(z)\, c_1\, c_2$

s_2 ist rücksetzbar – $s_2 \in RC$

d.h. T_2 wird zwingend abgebrochen, wenn T_1 abbricht

Vermeidung kaskadierender Abbrüche – ACA

- Schedule s vermeidet kaskadierende Abbrüche "**Schneeballeffekt**" (**A**voiding **C**ascading **A**borts), falls folgende Bedingung erfüllt ist:
  - $\forall_{i \neq j, v}\ w_j(v) \rightarrow_s r_i(v) \Rightarrow c_j \rightarrow_s r_i(v)$
  - T_i liest Wert v von T_j
  - dann muss Commit von T_j vor dem read sein
- Anders ausgedrückt:
  - Eine Transaktion darf nur Daten lesen, die zuletzt von einer bereits abgeschlossenen Transaktion geschrieben wurden. (⇨ Dirty Read wird verhindert)

Vermeidung kaskadierender Abbrüche – ACA

Beispiel

- Schedule $s_2 = w_1(x)\, w_1(y)\, r_2(u)\, w_2(x)\, r_2(y)\, w_2(y)\, w_1(z)\, c_1\, c_2$ des letzten Beispiels gehört nicht in die Klasse ACA

- $s_3 = w_1(x)\, w_1(y)\, r_2(u)\, w_2(x)\, w_1(z)\, c_1\, r_2(y)\, w_2(y)\, c_2$

- s_3 vermeidet kaskadierende Abbrüche:
  - daher gilt: $s_3 \in ACA$

Striktheit – ST

- Probleme mit **Before-Images**

T_1 bricht unkontrolliert ab

$s_3 = w_1(x)\, w_1(y)\, r_2(u)\, w_2(x)$ ↯ $w_1(z)\, c_1\ r_2(y)\, w_2(y)\, c_2$

Zeit ↓

DB-Inhalt	Operation
x = 1 (Anfangswert)	
x = 2	**$w_1(x{\leftarrow}2)$ [$BF_{x,T1}$ = 1]**
x = 3	**$w_2(x{\leftarrow}3)$ [$BF_{x,T2}$ = 2]**
	a_1 Rücksetzen von $w_1(x{\leftarrow}2)$ mit BF_x:=1. Überschreiben durch T_2 muss erhalten bleiben!
x = 3	
	a_2 Rücksetzen von $w_2(x{\leftarrow}3)$ mit BF_x:=??

Striktheit – ST

- Schedule s heißt strikt (**ST**rict), falls folgende Bedingung gilt:
  - $\forall_{i \neq j, v} w_j(v) \rightarrow_s o_i(v) \Rightarrow a_j \rightarrow_s o_i(v) \vee c_j \rightarrow_s o_i(v)$
  - Falls T_i von T_j den Wert v liest **oder überschreibt**
  - muss T_j vorher abgeschlossen sein (durch `abort` oder `commit`)
- Anders ausgedrückt:
  - Es darf kein geändertes Datenobjekt einer noch nicht beendeten Transaktion gelesen oder **überschrieben** werden.

Striktheit – ST

Beispiel

- $s_3 = w_1(x)\, w_1(y)\, r_2(u)\, w_2(x)\, \lightning a_1\, w_1(z)\, c_1 r_2(y)\, w_2(y)\, c_2$

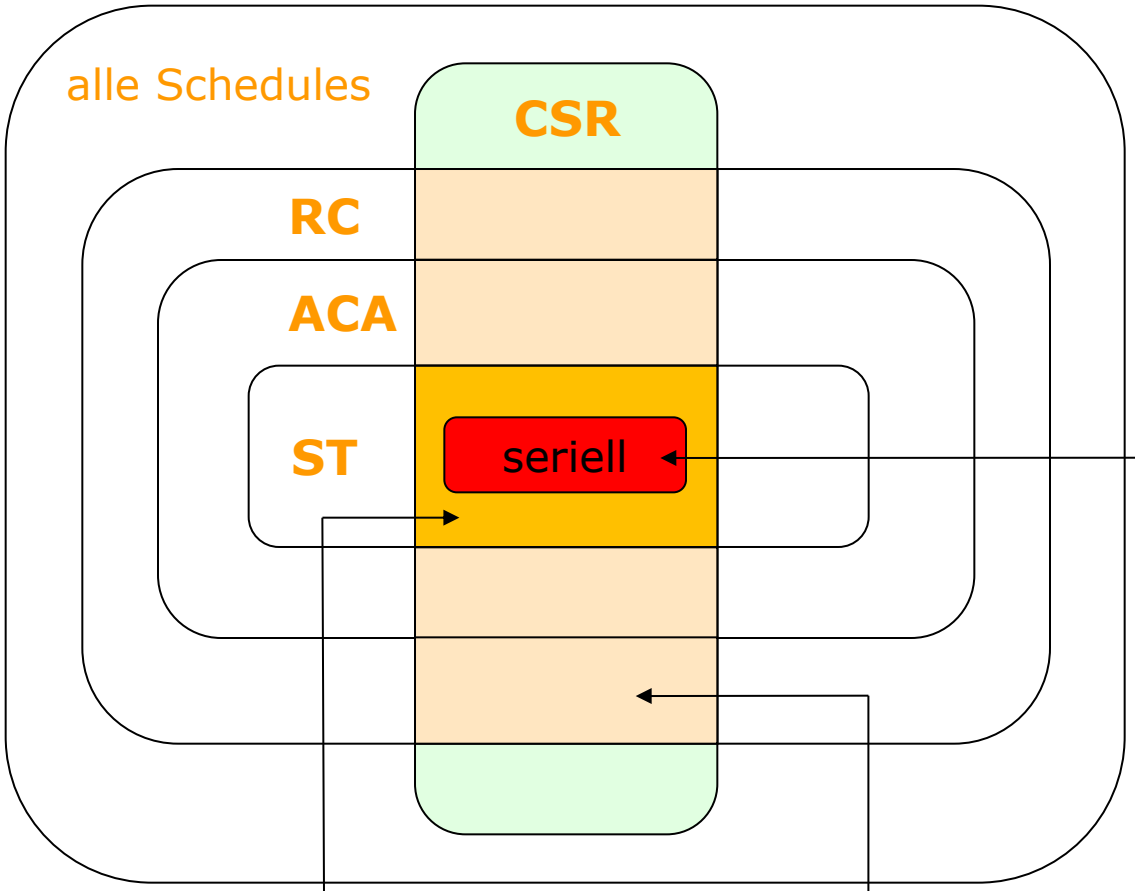
- $s_3 \notin$ **ST**
  - da $w_2(x)$ nach $w_1(x)$ aber vor a_1

- $s_4 = w_1(x)\, w_1(y)\, r_2(u)\, w_1(z)\, c_1\ w_2(x)\, r_2(y)\, w_2(y)\, c_2$

- $s_4 \in$ **ST**
  - da nun zuerst T_1 abgeschlossen wurde
  - bevor T_2 darauf zugreift

Serialisierbarkeit und Fehlersicherheit

Schedule-Klassen



CSR ... konfliktserialisierbare Schedules
RC ... rücksetzbare Schedules
ACA ... Schedules ohne kaskadierendes Rücksetzen
ST ... strikte Schedules

seriell ⊆ ST ⊆ ACA ⊆ RC
CSR orthogonal (also unabhängig)

nicht erstrebenswert
(geringere Performance)

ideal, oft durch strenge Sperrprotokolle

akzeptabel (durch Kaskadieren reparierbar)

Serialisierbarkeit und Fehlersicherheit

Probleme der Überprüfung

- **Prüfung** eines Schedule auf **Serialisierbarkeit / Fehlersicherheit** ist in der Praxis **schwierig** und **aufwändig**
  - Serialisierbarkeit von Ausführungsplänen nur im nachhinein überprüfbar (Grundidee = Entdeckung von Fehlern)
  - Zur Laufzeit nur **unvollständige Ausführungspläne** verfügbar ⇨ Überwachung unvollständiger Ausführungspläne notwendig
  - Transaktionen, die noch kein `commit` gemacht haben, können noch **jederzeit abgebrochen** werden

Serialisierbarkeit und Fehlersicherheit

Verfahren im Überblick

- Pessimistische Verfahren
  - Sperrbasierte Verfahren
  - Nicht-sperrbasierte Verfahren
    - Zeitstempel Verfahren
      - » erzwingen eine korrekte Reihenfolge der Bearbeitungsschritte mittels geeigneter Markierungen („Timestamps“) der Transaktionen
    - Serialisierbarkeitstester
      - » verwalten einen Konfliktgraphen und realisieren daher eine direkte Überprüfung der Konfliktserialisierbarkeit
- Optimistische Verfahren
  - Zertifikatoren
    - führen zunächst die komplette Transaktion aus und versuchen erst am Ende festzustellen, ob die Transaktion gemäß Serialisierbarkeit abgelaufen ist
    - 3 Phasen: Transaktion in privatem Workspace ausführen – Validierung auf CSR – falls OK merge mit DB, sonst abort
- Hybrid
  - verschiedene Verfahren für unterschiedliche Konflikt-Arten
- Multi-Version Concurrency Control (MVCC)
  - Transaktionen arbeiten auf verschiedenen Datenversionen
  - Meist in Verbindung mit pessimistischen Verfahren verwendet

Fokus dieser LVA

Review-Fragen

- Fehler bei Mehrbenutzerbetrieb?
- Read/Write Modell?
- Schedule?
- Effektäquivalenz?
- Konfliktmatrix?
- Konfliktserialisierbarkeit?
- Fehlersicherheit?
- Serialisierbarkeit & Fehlersicherheit

Überblick

- Motivation
- Transaktionsverwaltung
- Mehrbenutzersynchronisation
  - Fehler bei Mehrbenutzerbetrieb
  - Serialisierbarkeit
  - Fehlersicherheit
  - Sperrbasierte Verfahren
  - Nicht-sperrbasierte Verfahren
  - Multi-Version Concurrency Control (MVCC)
- Transaktionen in SQL
- Zusammenfassung

© 2026

Sperrbasierte Verfahren

- Sperrende (pessimistische) Verfahren stellen während des laufenden Betriebs sicher, dass der resultierende Ausführungsplan serialisierbar bleibt.

- **Bevor** eine Transaktion eine **Operation** ausführen darf,
  - muss sie eine entsprechende **Sperre** für das betreffende Datenbankobjekt erhalten haben.

- **Sperrprotokoll**
  - = Vorschrift bzgl. des Setzens von Sperren
  - garantiert Konfliktserialisierbarkeit ohne zusätzliche Tests!
  - einfach zu implementieren, geringer Laufzeitaufwand!

Sperrbasierte Verfahren
Sperrmodell

- Für Transaktionen werden drei zusätzliche Operationen definiert:
  - **Schreib**- und **Lesesperren** in folgender Notation:
    - `rl(x)`: Lesesperre (read lock) auf einem Objekt `x`
    - `wl(x)`: Schreibsperre (write lock) auf einem Objekt `x`
  - **Entsperren** `ru(x)` und `wu(x)`, oft zusammengefasst `u(x)`
- Für das Setzen von Sperren auf dasselbe DB-Objekt von mehreren Transaktionen gilt folgende Verträglichkeitsmatrix:

T_j / T_i	$rl_i(x)$	$wl_i(x)$
$rl_j(x)$	✓	✗
$wl_j(x)$	✗	✗

Bestehende Sperre

← Angeforderte Sperre

Angeforderte Sperre wird gewährt (✓) oder nicht gewährt (✗)

Sperrbasierte Verfahren

Sperrdisziplin

- Schreibzugriff **`w(x)`**
  - nur nach Setzen einer Schreibsperre **`wl(x)`** möglich
- Lesezugriffe **`r(x)`**
  - nur nach **`rl(x)`** oder **`wl(x)`** erlaubt
- Nur Objekte sperren,
  - die **nicht bereits** von einer anderen Transaktion **gesperrt** sind
- Nach **`rl(x)`**
  - nur noch **`wl(x)`** erlaubt,
  - danach auf **`x`** keine Sperre mehr;
  - Sperren derselben Art werden maximal einmal gesetzt
- Bei Transaktionsende - `COMMIT`
  - müssen alle Sperren der Transaktion aufgehoben werden

Sperrbasierte Verfahren

Sperrdisziplin vs. Serialisierbarkeit

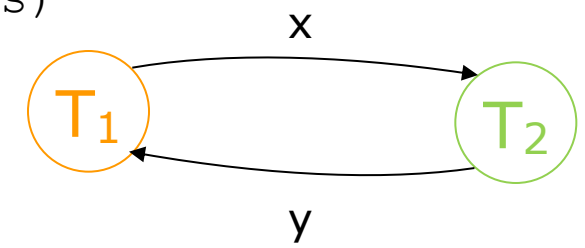
Zeit

T_1	T_2
wl(x)	
w(x)	
u(x)	
	wl(x)
	w(x)
	u(x)
	wl(y)
	w(y)
	u(y)
wl(y)	
w(y)	
u(y)	

⇨ nicht konfliktserialisierbar!

Konfliktgraph:

G(s)

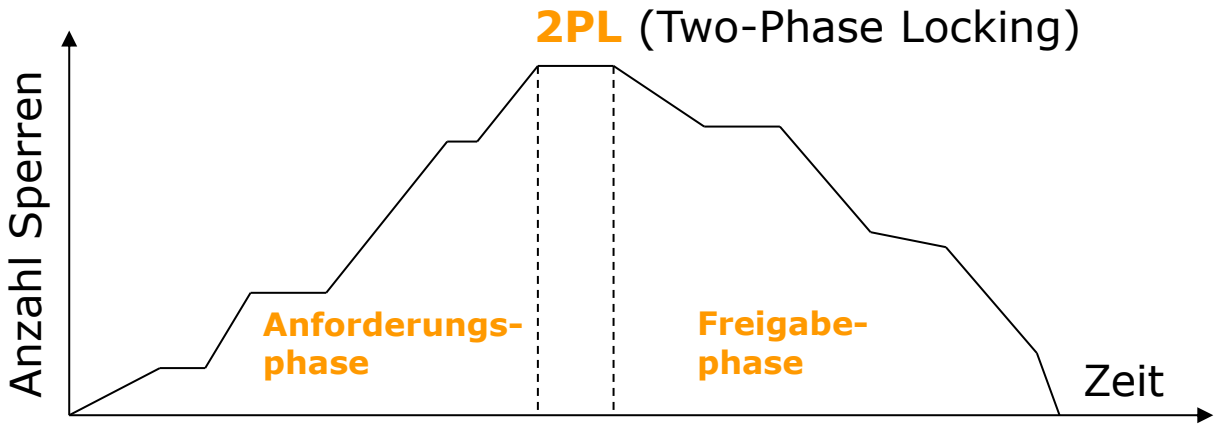


⇨ Sperrprotokolle notwendig

Sperrbasierte Verfahren

Zwei-Phasen-Sperrprotokoll (2PL)

- Jede Transaktion durchläuft **zwei Phasen**:
  - **Anforderungsphase**, in der Sperren angefordert werden, aber keine freigegeben werden dürfen
  - **Freigabephase**, in der bisher erworbene Sperren freigegeben werden, aber keine neunen angefordert werden dürfen
- Bei **Transaktionsende** müssen alle erworbenen Sperren zurückgegeben werden



- 2PL garantiert serialisierbaren Ablauf paralleler Transaktionen
  - verhindert aber weder das Phantom-Problem noch Deadlocks

Sperrbasierte Verfahren

Zwei-Phasen-Sperrprotokoll - Beispiel

- **T1 modifiziert** nacheinander die Datenobjekte x und y (z.B. eine Überweisung)
- **T2 liest** nacheinander dieselben Datenobjekte x und y (z.B. zur Aufsummierung der beiden Kontostände)

Schritt	T_1	T_2	Bemerkung	Locks x	Locks y
1.	wl(x)			T1(w)	
2.	r(x)			T1(w)	
3.	w(x)			T1(w)	
4.		rl(x)	T_2 muss warten	T1(w)	
5.	wl(y)			T1(w)	T1(w)
6.	r(y)			T1(w)	T1(w)
7.	u(x)		T_2 wecken	T2(r)	T1(w)
8.		r(x)		T2(r)	T1(w)
9.		rl(y)	T_2 muss warten	T2(r)	T1(w)
10.	w(y)			T2(r)	T1(w)
11.	u(y)		T_2 wecken	T2(r)	T2(r)
12.		r(y)		T2(r)	T2(r)
13.	commit			T2(r)	T2(r)
14.		u(x)			T2(r)
15.		u(y)			
16.		commit			

Verwaltung einer Sperrtabelle durch das DBMS

Sperrbasierte Verfahren

Zwei-Phasen-Sperrprotokoll – Kaskadierendes Rücksetzen

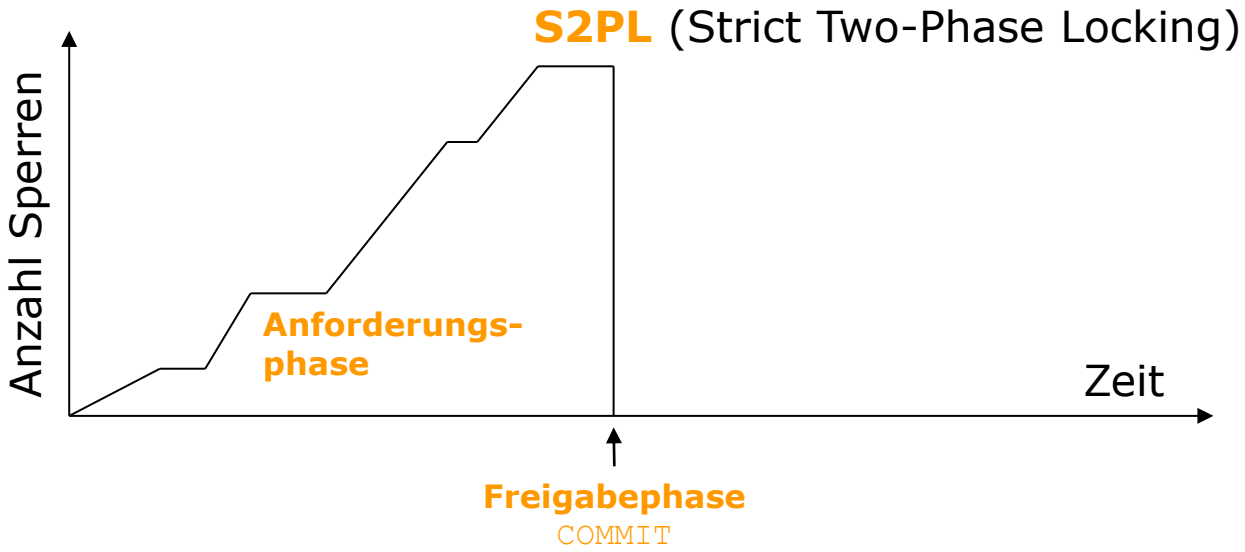
- Was passiert, wenn **T1 direkt vor Schritt 13 scheitert**?
- Dann muss **T2 zurückgesetzt** werden, da T2 "schmutzige" Daten von T1 gelesen hat!

Schritt	T_1	T_2	Bemerkung
1.	wl(x)		
2.	r(x)		
3.	w(x)		
4.		rl(x)	T_2 muss warten
5.	wl(y)		
6.	r(y)		
7.	u(x)		T_2 wecken
8.		r(x)	
9.		rl(y)	T_2 muss warten
10.	w(y)		
11.	u(y)		T_2 wecken
12.		r(y)	
13.	abort		Kaskadierender Abbruch von T_2
14.		u(x)	
15.		u(y)	
16.		commit	

Sperrbasierte Verfahren

Zwei-Phasen-Sperrprotokoll – Striktes 2PL

- Vermeidet kaskadierendes Rücksetzen!
  - Alle **Sperren** werden bis zum **Transaktionsende** gehalten
  - damit ist kaskadierendes Rücksetzen ausgeschlossen



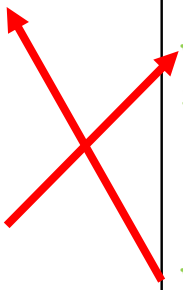
- Striktes 2PL ist die häufigste Variante in DMBS

Sperrbasierte Verfahren

Zwei-Phasen-Sperrprotokoll – Verklemmungen (Deadlocks)

- Sperren können Deadlocks verursachen:

Schritt	T_1	T_2	Bemerkung
1.	wl(x)		
2.		wl(y)	
3.		r(y)	
4.	r(x)		
5.	w(x)		
6.	wl(y)		T_1 muss warten auf T_2
7.		wl(x)	T_2 muss warten auf T_1
8.	…	…	⇨ DEADLOCK ORACLE ERROR at line 1: ORA-00060: deadlock detected while waiting for resource

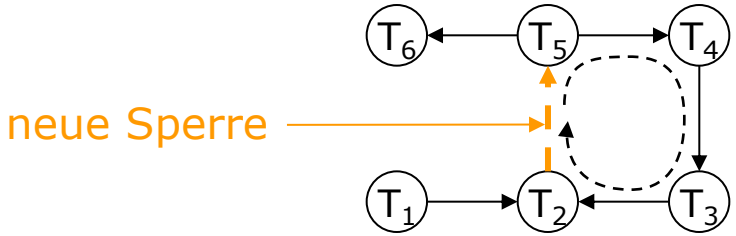


- Deadlocks - was tun?
  - **Erkennen** und **Auflösen** von Deadlocks
    - ⇨ Analyse von Wartegraphen
    - ⇨ Timeout-Methode
  - Deadlocks im vorhinein **Vermeiden**
    - ⇨ Konservatives 2PL-Protokoll (Preclaiming)
    - ⇨ Zeitstempel-basiert

Sperrbasierte Verfahren

Verklemmungen Erkennen + Auflösen mit Wartegraph

- Wartegraph (= Wartebeziehungen zw. Transaktionen):



  - Deadlock liegt dann vor, wenn der Wartegraph einen Zyklus hat.
  - Auflösen durch Abbruch einer Transaktion bzw. einer Menge von Transaktionen.
- Auswahlkriterien
  - **Länge** einer Transaktion
    - Transaktion läuft seit langer Zeit
  - **Rücksetzaufwand** einer Transaktion
    - bereits viele Aktualisierungen ausgeführt
  - **Wichtigkeit** einer Transaktion
  - **Anzahl** der aufgebrochenen **Zyklen**
    - wie viele Zyklen können aufgebrochen werden, wenn eine bestimmte Transaktion für den den Abbruch ausgewählt wird.

Sperrbasierte Verfahren

Verklemmungen Erkennen + Auflösen mit Timeout

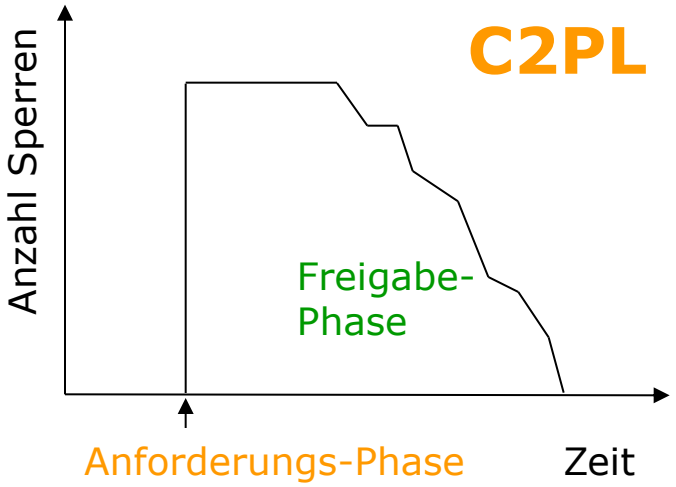
- Ausführung einer Transaktion wird überwacht
- Wenn nach einer **Zeitspanne kein Fortschritt**, geht man von einem Deadlock aus und setzt die Transaktion zurück
- Herausforderung: **Zeitspanne richtig wählen**
  - "zu kurz" ⇨ zu viele unnötige Rollbacks von nicht verklemmten Transaktionen
  - "zu lang" ⇨ existierende Deadlocks bleiben lange im System
    - verringern Performance
    - können (im Extremfall) gesamtes System blockieren.

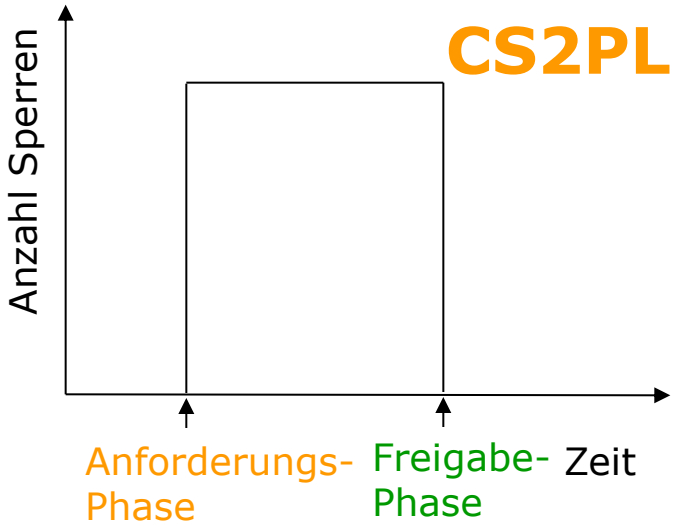
- **Timeout-Methode** in der Praxis sehr verbreitet
  - einfache Implementierung

Sperrbasierte Verfahren

Verklemmungen Vermeiden mit Konservativen 2PL

- Transaktionen werden erst dann begonnen, wenn alle Sperranforderungen beim Transaktionsbeginn erfüllt sind (Preclaiming)





- Vermeidet Deadlocks!
  - **ABER:**
    - Wie weiß die Transaktion vorher (bspw. in interaktiven Anwendungen), welche DB-Objekte sie benötigt?
    - Deswegen in der Praxis **nicht realisierbar**!

Sperrbasierte Verfahren
Livelock-Problem

- Es ist möglich, dass eine Transaktion eine Sperre anfordert, diese aber nie bekommt, weil andere Transaktion vorgezogen werden (Starvation, Livelock)

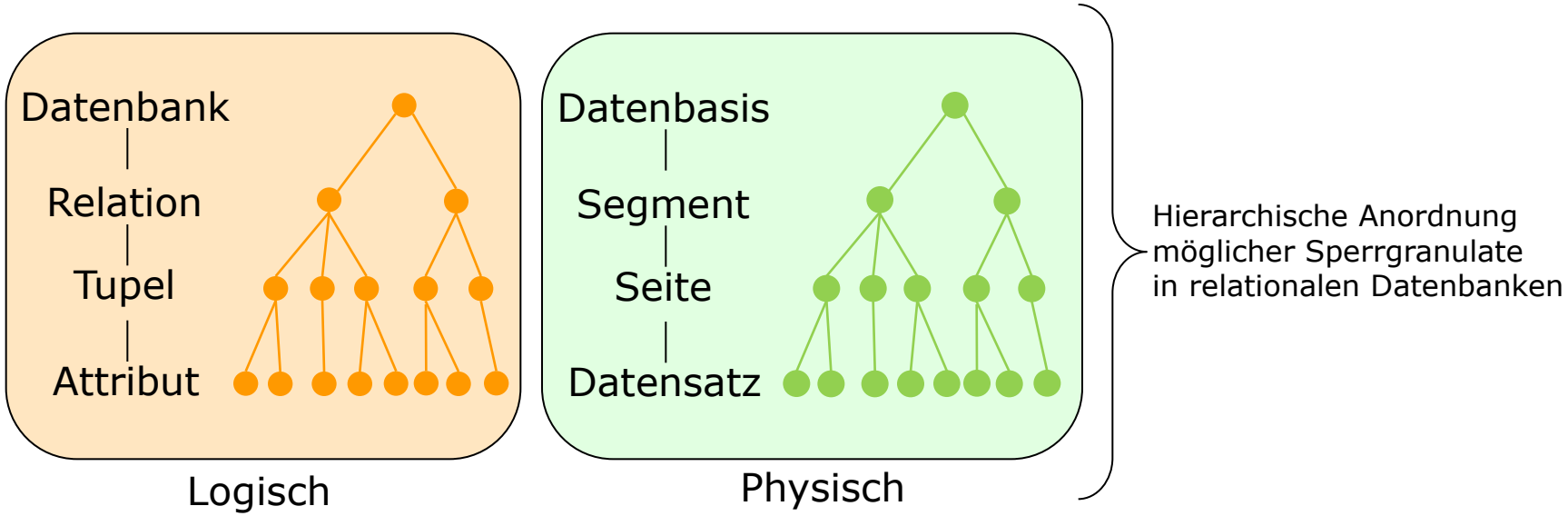
1. T1 sperrt A
2. T2 will A sperren, muss aber warten
3. T3 will danach A sperren, muss auch warten
4. T1 gibt A frei
5. T3 kommt vor T2 an eine Zeitscheibe, sperrt A
6. T2 will weiterhin A sperren, muss aber warten
7. T4 will danach A sperren, muss auch warten
8. T3 gibt A frei
9. T4 kommt vor T2 an die nächste Zeitscheibe ...

- Abhilfe:
  - **First-Come-First-Serve-Strategie**

Sperrbasierte Verfahren

Hierarchische Sperrverfahren - Sperrgranulate

- **Sperrgranulat** = Größe des zu sperrenden Objektes
- Bislang alle Sperren (`rl`, `wl`) auf **derselben Hierarchiestufe** (z.B. Tupel)



Sperrbasierte Verfahren

Hierarchische Sperrverfahren – Flexible Sperrgranularität

- Einschränkung auf nur eine Sperrgranularität hat für alle Transaktionen Nachteile:
  - Zu **kleine Granularität** belastet Transaktionen mit hohem Datenzugriff, da sie viele Sperren anfordern müssen.
  - Zu **große Granularität** schränkt den Parallelitätsgrad des Systems unnötig ein, da Datenobjekte implizit gesperrt werden, die gar nicht benötigt werden.

- Lösung - Flexible Sperrgranularität:
  - Einführung **zusätzlicher Sperrmodi**, welche eine flexible Auswahl eines bestimmten Sperrgranulats pro Transaktion unterstützen. Sperrgranulat bestimmt Parallelität/Aufwand.
  - **Intensionale Sperren** – Anwartschaftssperre (I-Sperre)

© 2026

Sperrbasierte Verfahren

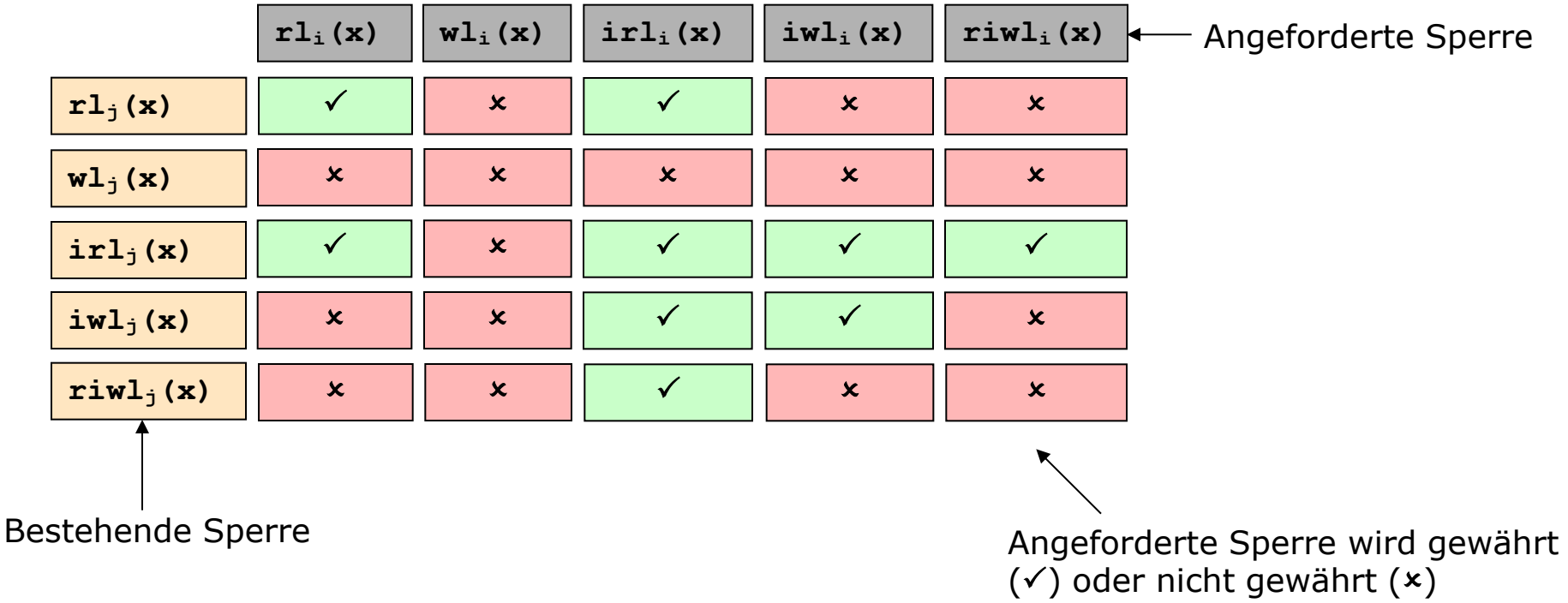
Hierarchische Sperrverfahren – Intentsionale Sperre

- Zeigen auf **höherer Ebene** der Granularitätshierarchie die „**Absicht**“ einer weiter **unten** in der Hierarchie gesetzten Sperre an

- **Intensionale Lesesperre** (`irl`)
  - Weiter unten in der Hierarchie ist eine Lesesperre (`rl`) beabsichtigt
- **Intensionale Schreibsperre** (`iwl`)
  - Weiter unten in der Hierarchie ist eine Schreibsperre (`wl`) beabsichtigt
- **Lese- und intensionale Schreibsperre** (`riwl`)
  - Setzt eine Lesesperre (`rl`) auf der aktuellen Hierarchiestufe und zeigt an, dass weiter unten in der Hierarchie eine Schreibsperre (`wl`) beabsichtigt ist

Sperrbasierte Verfahren

Hierarchische Sperrverfahren – Konfliktmatrix



- In welcher **Reihenfolge** sollen die Sperren gesetzt bzw. freigegeben werden?

Sperrbasierte Verfahren

Hierarchische Sperrverfahren – Ablaufregel

- Sperrung eines Datenobjektes muss so durchgeführt werden, dass erst geeignete Sperren in allen übergeordneten Knoten in der Hierarchie erworben werden.
  - Sperrung erfolgt "top down", d.h. Sperren pflanzen sich nach unten (in Richtung der Blätter) fort
  - Sperrfreigabe erfolgt "bottom up"

Sperrbasierte Verfahren

Hierarchische Sperrverfahren – Ablauf

1. Sperren werden auf einem **Pfad** in der Reihenfolge von der **Wurzel** zum **Zielobjekt** gesetzt
2. DB-Objekt, auf dem **gearbeitet** werden soll, wird mit **herkömmlicher** Schreib- oder Lesesperre **gesperrt**
3. Alle **anderen Knoten** auf dem Pfad bekommen **intentionale Sperren**
4. Sperren können **verschärft** werden,
   - das heißt ein `rl` kann zum `wl` werden
   - ein `irl` zum `rl` und
   - ein `irl` zum `iwl`
5. Freigabe erfolgt in umgekehrter Reihenfolge

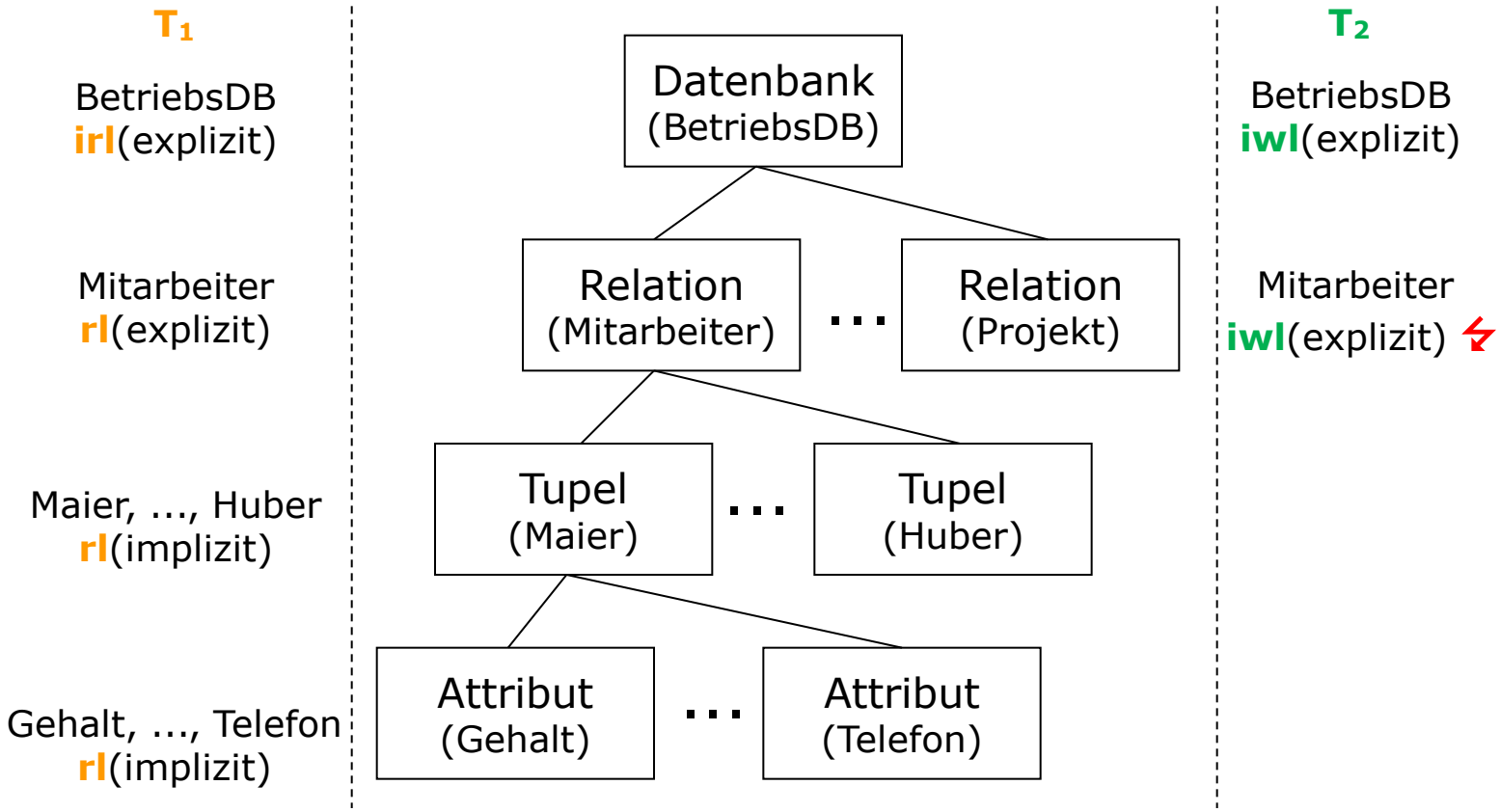
© 2026

Sperrbasierte Verfahren

Hierarchische Sperrverfahren – Beispiel 1

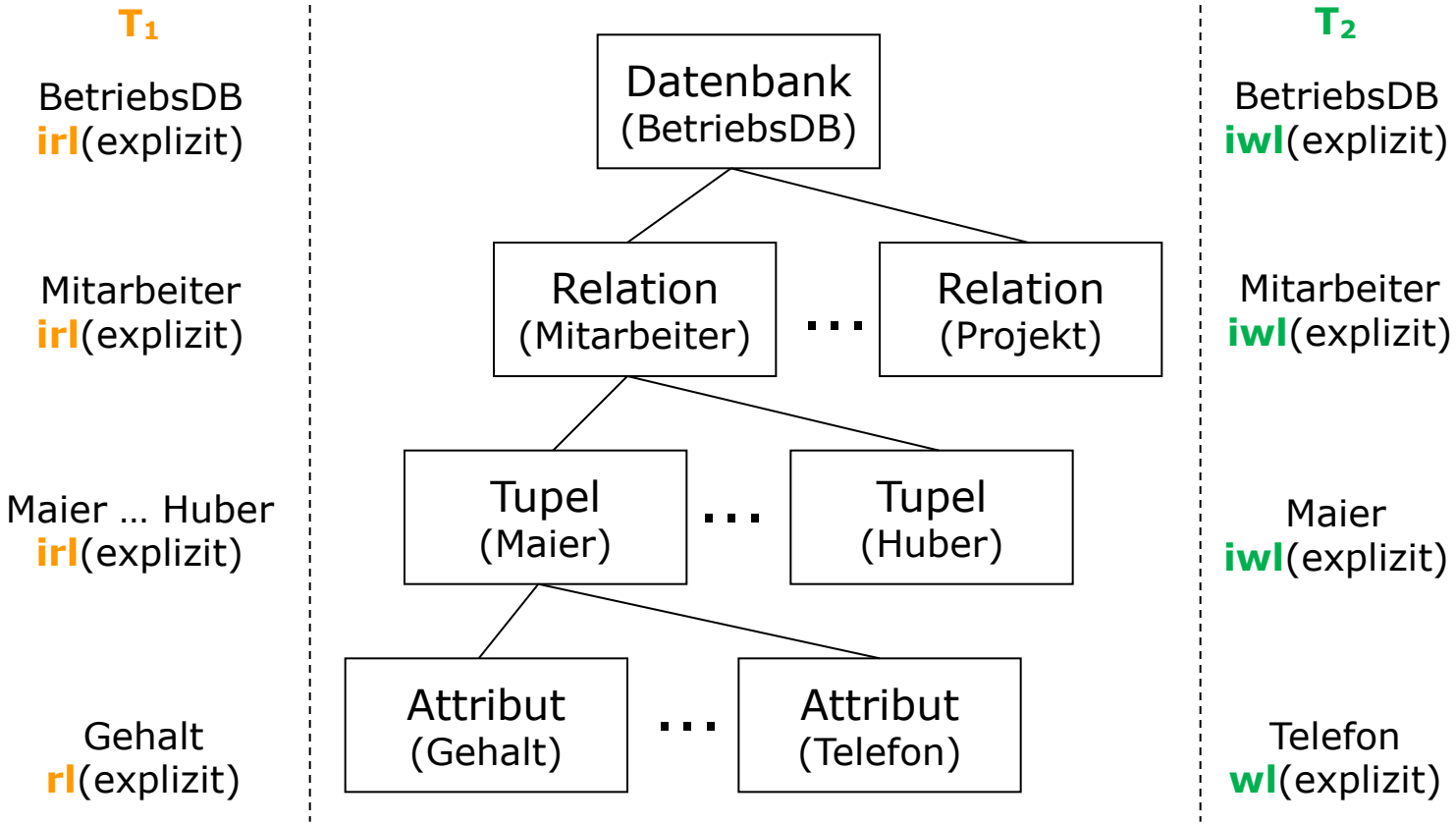
T_1: berechnet Durchschnittsgehalt aller Mitarbeiter

T_2: Ändert Telefonnummer von Mitarbeiter Maier



Sperrbasierte Verfahren

Hierarchische Sperrverfahren – Beispiel 2

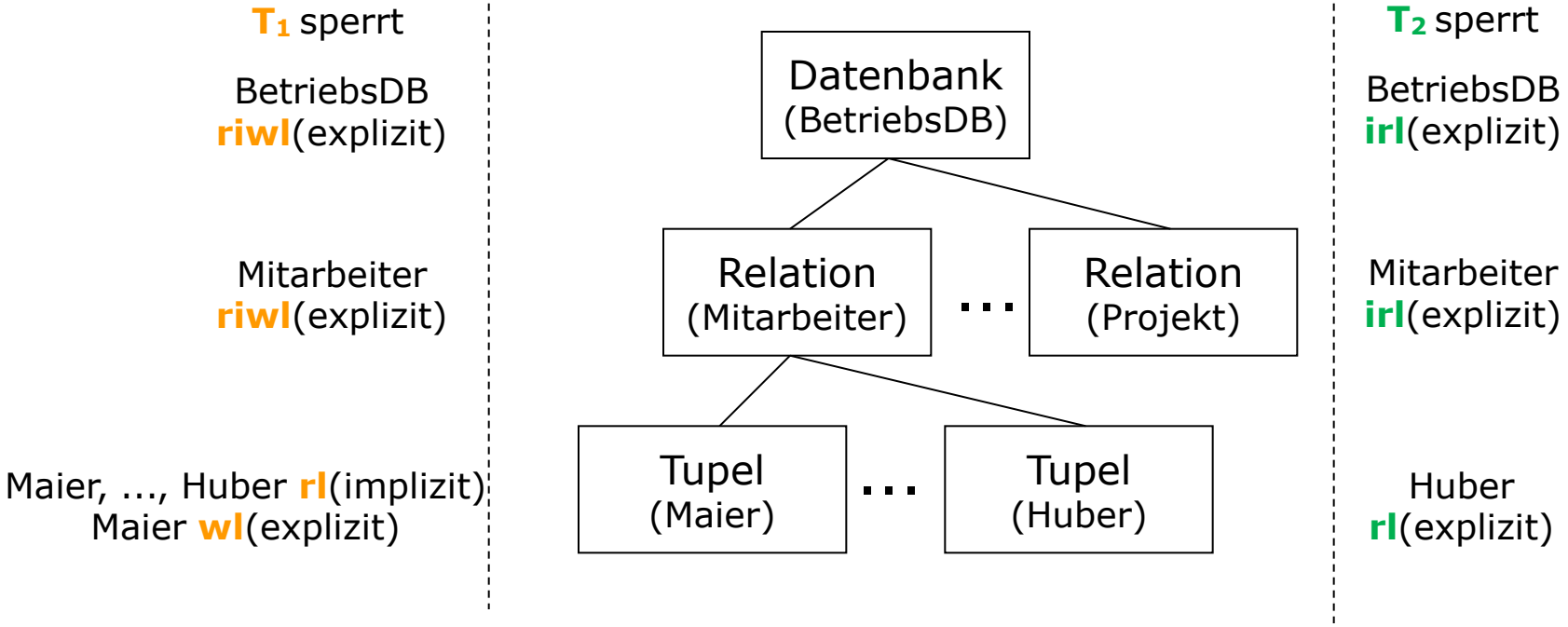


Sperrbasierte Verfahren

Hierarchische Sperrverfahren – Beispiel 3 (riwl)

T_1: passt Gehalt des teuersten Mitarbeiters an das Gehalt des zweitteuersten Mitarbeiters an

T_2: liest alle Daten des Mitarbeiters Huber



Sperrbasierte Verfahren

Hierarchische Sperrverfahren – Zusammenfassung

- Ermöglichen Transaktionen mit **wenigen Zugriffen** pro Granularitätsstufe
  - auf niedriger Hierarchiestufe (= kleine Granularität) zu sperren
  - ⇨ höherer **Parallelitätsgrad**

- Ermöglichen Transaktionen mit **vielen Zugriffen** pro Granularitätsstufe
  - auf entsprechend höherer Hierarchiestufe (= große Granularität) zu sperren
  - ⇨ geringerer **Sperraufwand**

- **Lock Escalation**
  - sobald eine bestimmte Anzahl von Sperren in der kleineren Granularität erworben wurde, wird automatisch auf die nächsthöhere Granularität umgeschaltet

© 2026

Überblick

- Motivation
- Transaktionsverwaltung
- Mehrbenutzersynchronisation
  - Fehler bei Mehrbenutzerbetrieb
  - Serialisierbarkeit
  - Fehlersicherheit
  - Sperrbasierte Verfahren
  - Nicht-sperrbasierte Verfahren
  - Multi-Version Concurrency Control (MVCC)
- Transaktionen in SQL
- Zusammenfassung

© 2026

Nicht-sperrbasierte Verfahren

- Nicht-sperrende Verfahren versuchen, Konflikte bei der verschränkten Ausführung von Transaktionen zu erkennen und aufzulösen.
  - Einfache Auflösung durch Zurücksetzen von Transaktionen
  - Keine Deadlocks
  - Potentiell höhere Parallelität als bei Sperrverfahren

- **Zeitstempel-basierte** Verfahren
  - erzwingen eine korrekte Reihenfolge der Bearbeitungsschritte mittels geeigneter Markierungen der Transaktionen
- **Serialisierungsgraph-Tester**
  - verwalten einen Konfliktgraphen und realisieren daher eine direkte Überprüfung der Konfliktserialisierbarkeit

Nicht-sperrbasierte Verfahren

Zeitstempel-basierende Verfahren

- Bei **Konflikt** wird Transaktion, die zu dem Konflikt geführt hat, **abgebrochen**
  - lange Transaktionen bearbeiten ihre späteren Schritte längere Zeit nach der Vergabe der Zeitmarke
  - kommen damit häufiger "zu spät" und müssen abgebrochen werden

- Zum Erkennen von Konflikten werden **Transaktionen** und **Datenobjekten Zeitstempel** zugeordnet
  - Jede Transaktion erhält als eindeutigen Zeitstempel den Zeitpunkt des Transaktionsbeginns
  - Jedem Datenobjekt werden zwei Marken zugeordnet:
    - $t_r(O)$ Zeitstempelwert der jüngsten Transaktion, die O gelesen hat
    - $t_w(O)$ Zeitstempelwert der jüngsten Transaktion, die O geschrieben hat

Nicht-sperrbasierte Verfahren

Zeitstempel-Bedingungen

- Stellen sicher, dass die Abarbeitung der Transaktionen in Zeitstempel-Reihenfolge **äquivalent** zu einer **seriellen Ausführung** ist:

  - T_1 mit Zeitstempel t_1 will O **lesen**, also $r_1(O)$
    T_1 mit Zeitstempel t_1 darf ein Datenobjekt mit Schreibstempel $t_w(O)$ nicht lesen, wenn $\mathbf{t_w(O) > t_1}$ gilt, also das Objekt bereits von einer jüngeren Transaktion verändert wurde.
    ⇨ Transaktion T_1 muss zurückgesetzt werden

  - T_1 mit Zeitstempel t_1 will O **schreiben**, also $w_1(O)$
    T_1 mit Zeitstempel t_1 darf ein Datenobjekt mit Lese-Zeitstempel $t_r(O)$ nicht verändern, wenn
    - $\mathbf{t_r(O) > t_1}$ gilt, also das Objekt bereits von einer jüngeren Transaktion gelesen wurde oder
    - $\mathbf{t_w(O) > t_1}$ gilt, also das Objekt bereits von einer jüngeren Transaktion überschrieben wurde

    ⇨ Transaktion T_1 muss zurückgesetzt werden

Nicht-sperrbasierte Verfahren

Zeitstempel-Verfahren – Basis-Algorithmus

INPUT: Operation X der Transaktion T auf einem Datenobjekt O
OUTPUT: Lese- oder Schreibstempel von O oder Abbruch von T

```
(1) if X = READ and t ≥ t_w
    then READ
         if t > t_r
         then t_r := t endif;
    endif;
```

```
(2) if X = WRITE and t ≥ t_r and t ≥ t_w
    then WRITE
         t_w := t;
    endif;
```

```
(3) if X = WRITE and t_r ≤ t < t_w
    then {do nothing} endif;
```

Regel für Optimiertes Zeitstempel-Verfahren! (Blindes Schreiben)

```
(4) if (X = READ and t < t_w) or (X = WRITE and t < t_r)
    then abort(T) endif;
```

Nicht-sperrbasierte Verfahren

Zeitstempel-Verfahren – Beispiel

T_1	T_2	T_3	A		B		C	
			$t_r(A)$	$t_w(A)$	$t_r(B)$	$t_w(B)$	$t_r(C)$	$t_w(C)$
$t_s = 200$	$t_s = 150$	$t_s = 175$	0	0	0	0	0	0
read B			**0**	**0**	**200**	**0**	**0**	**0**
	read A		**150**	**0**	**200**	**0**	**0**	**0**
		read C	**150**	**0**	**200**	**0**	**175**	**0**
write B			**150**	**0**	**200**	**200**	**175**	**0**
write A			**150**	**200**	**200**	**200**	**175**	**0**
	write C ↯		**150**	**200**	**200**	**200**	**175** ↯	**0** ↯
	abort		**150**	**200**	**200**	**200**	**175**	**0**
		write A	**150** ✓	**200** ↯	**200**	**200**	**175**	**0**

T_2 muss mit $t_s > 200$ neu gestartet werden.

Das letzte Schreiben auf A war blind ($w1(A)$) und kann beibehalten werden, ohne dass Transaktion abgebrochen werden muss (siehe (3) in Basis-Algorithmus)

↯ = entdeckter Konflikt

Nicht-sperrbasierte Verfahren

Zeitstempel-Verfahren – Livelocks

T_1	T_2	T_1'	T_2'	A		B	
				mrs	mws	mrs	mws
t_s= 100	t_s= 110	t_s= 120	t_s= 130	0	0	0	0
write B				**0**	**0**	**0**	**100**
	write A			**0**	**110**	**0**	**100**
read A ↯				**0**	**110** ↯	**0**	**100**
		write B		**0**	**110**	**0**	**120**
	read B ↯			**0**	**110**	**0**	**120** ↯
			write A	**0**	**130**	**0**	**120**
		read A ↯		**0**	**130** ↯	**0**	**120**

Nicht-sperrbasierte Verfahren

Zeitstempel-Verfahren – Vor- und Nachteil

- **Nachteil:** Anfälligkeit gegen stark variierende Laufzeiten von Transaktionen
  - lang andauernde Transaktionen bearbeiten ihre letzten Schritte längere Zeit nach der Vergabe der Zeitmarke
  - kommen damit mit diesen Schritten mit größerer Wahrscheinlichkeit **zu spät** und
  - erzeugen viele Rücksetzungen

- **Vorteil:** einfache Realisierung in verteilten Systemen

Überblick

- Motivation
- Transaktionsverwaltung
- Mehrbenutzersynchronisation
  - Fehler bei Mehrbenutzerbetrieb
  - Serialisierbarkeit
  - Fehlersicherheit
  - Sperrbasierte Verfahren
  - Nicht-sperrbasierte Verfahren
  - Multi-Version Concurrency Control (MVCC)
- Transaktionen in SQL
- Zusammenfassung

Multiversionen-Synchronisation
MVCC

- Prinzip:
  - **Reduzierung der Konflikte**, wenn zu den geänderten Datenobjekten **mehrere Versionen** (Mehrversionen-Scheduler) geführt werden.
  - Jede Transaktion bekommt eine eigene **Momentaufnahme** (lesekonsistente Sicht) der Datenobjekte zu sehen.

- Gegebener Schedule s:

  Dirty Read

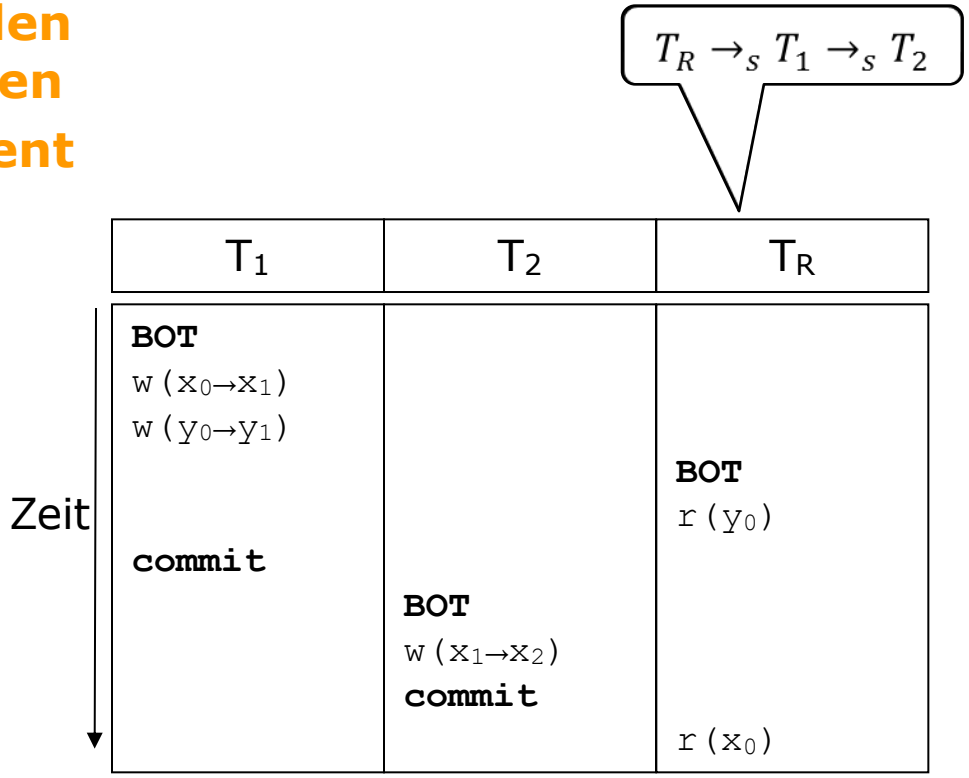
  $s = r_1(x)\ w_1(x)\ r_2(x)\ \mathbf{w_2(y)}\ \mathbf{r_1(y)}\ w_1(z)\ c_1\ c_2$

  - $s \notin CSR$
  - aber tolerierbar,
    - wenn $r_1(y)$ gültige (= committed) Version von y liest, so dass $r_1(y)$ lesekonsistent zu $r_1(x)$ ist (y zu Beginn von s) und
    - WriteSets von T_1 und T_2 disjunkt sind: $\{w_1(x), w_1(z)\}\ \{w_2(y)\}$
  - dann:
    - kein Dirty Read mehr
    - s äquivalent zu $s' = T_1T_2$ bzw. T_2T_1

Multiversionen-Synchronisation

Prinzip

- Jede **Änderungsoperation** `w` erzeugt **neue Version** `w'` des Datenobjektes
  - Leseoperationen können **passende Version wählen** und **setzen keine Sperren**
  - Versionen sind **transparent** für Applikationen



Multiversionen-Synchronisation

Versionsverwaltung 1/2

- Aufgabe: Bestimmung der zu lesenden Version
  - globaler Transaktionszähler `TNC` (transaction number count)
  - Für Transaktionen `T` gibt es jeweils einen Commit-Zeitstempel `cts(T)` und `BOT`-Zeitstempel `bts(T)`
  - Schreibzeitstempel `wts(x)` für jede Version eines geänderten Objekts `x`

- Commit einer Änderungstransaktion T_U:
  - aktueller `TNC`-Wert wird als Commit-Zeitstempel $cts(T_U)$ genommen sowie Inkrementieren von `TNC`
  - modifiziertes Objekt: Schreibzeitstempel $wts(x) = cts(T_U)$

Multiversionen-Synchronisation

Versionsverwaltung 2/2

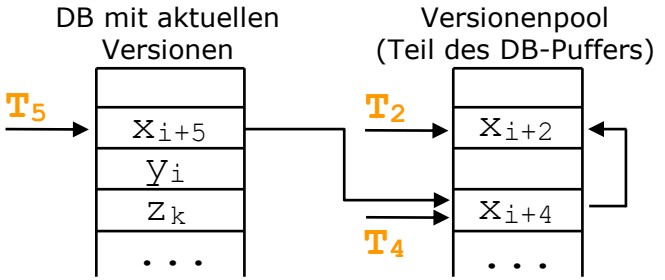
- Lesetransaktion T_R:
  - aktueller TNC-Wert als BOT-Zeitstempel: $bts(T_R) = TNC$
  - T_R hat Zugriff auf jüngste Version von x mit:

    $$wts(x) < bts(T_R)$$

    d.h. Lesen von x_i mit

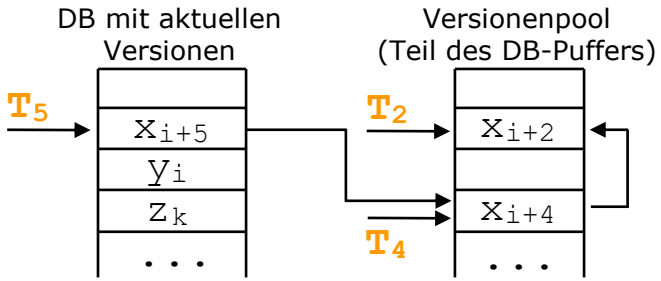
    $$i = max\{j \mid wts((x_j) < bts(T_R)\}$$
- Aufgabe: Freigabe nicht mehr benötigter Versionen (garbage collection)
  - BOT-Zeitstempel der ältesten Lesetransaktion: bts_min
  - Version x_i von Objekt x kann gelöscht werden, falls neuere Version x_j existiert, so dass gilt:

    $$wts(x_i) < wts(x_j) < bts_min$$
- Implementierung als Ringpuffer



Multiversionen-Synchronisation
Zusammenfassung

- Entkopplung von Lese- und Änderungsoperationen
  - ⇨ Lesetransaktion hat Sicht auf Datenbank wie bei `BOT`
- Keine Synchronisation von Lesetransaktionen notwendig
  - ⇨ Reduzierung der Konfliktwahrscheinlichkeit
  - ⇨ Erhöhung des Durchsatzes
- Synchronisation von Änderungsoperationen durch andere Verfahren
  - ⇨ Sperrverfahren, Zeitstempel
- zusätzlicher Speicher- und Wartungsaufwand
  - Versionenverwaltung, Garbage Collection
  - Auffinden von Versionen
  - Speicherplatzoptimierung mittels Komprimierungstechniken



- In kommerziellen DBS im Einsatz
  - Oracle, MS SQL Server, MySQL, PostgreSQL, …
  - NoSQL-Systemen, …

Review-Fragen

- Sperrbasierte Verfahren
  - [C][S]2PL
  - Deadlocks, Livelocks
  - Hierarchische Sperren
  - Intentionale Sperren
  - Lock Escalation
- Zeitstempelbasierte Verfahren
  - Wann muss rückgesetzt werden?
- MVCC

Überblick

- Motivation
- Transaktionsverwaltung
- Mehrbenutzersynchronisation
  - Fehler bei Mehrbenutzerbetrieb
  - Serialisierbarkeit
  - Fehlersicherheit
  - Sperrbasierte Verfahren
  - Nicht-sperrbasierte Verfahren
  - Multi-Version Concurrency Control (MVCC)
- Transaktionen in SQL
- Zusammenfassung

Transaktionen in SQL

- Transaktion beginnt implizit bei der ersten Datenbank-operation, die nach Beendigung der letzten Transaktion aufgerufen wird.

Beispiel

```
/* Begin of Transaction*/
Operation1;
...
Operation2;
COMMIT; /* Transaktion wird erst durch COMMIT beendet! */
```

- Transaktion kann mit **ROLLBACK** explizit abgebrochen werden; dann werden alle bisherigen Änderungen dieser Transaktion rückgängig gemacht.

Beispiel

```
Operation1;
...
Operation2;
ROLLBACK;
```

Isolation

- Isolation garantiert **"automatisch" Korrektheit**
- Realisierung (meist durch Sperren) muss **alle möglichen Operationen** (Lesen, Ändern, Einfügen, Löschen) in Betracht ziehen
- **Parallelitätsgrad** wird durch Sperren (Blockierungs-zeiten) **eingeschränkt**
- Je nach Anwendung
  - werden nur bestimmte Operationen genutzt,
  - ist Korrektheit vielleicht nicht erforderlich (z.B. nur lesender Zugriff für statistische Auswertungen),

  d.h. Parallelität wird unnötig eingeschränkt.

➢ **abgeschwächte, skalierbare Isolationsanforderungen**

Isolationsebenen in SQL

- **SQL** definiert verschiedene Isolationsebenen beim Lesen von Daten

Notation

```
SET TRANSACTION
  [ ( READ ONLY | READ WRITE ), ]      <-- Zugriffsmodus
  [ISOLATION LEVEL                     <-- Isolationsebene
    ( READ UNCOMMITTED |
      READ COMMITTED |
      REPEATABLE READ |
      SERIALIZABLE ), ]
  [...]
```

- Standardeinstellung:

```
SET TRANSACTION READ WRITE,
    ISOLATION LEVEL SERIALIZABLE;
```

Isolationsebenen in SQL

READ UNCOMMITTED

T_1	T_2
	read(A)
	. . .
	write(A)
read(A)	
. . .	
	rollback

- Lesen von nicht endgültig geschriebenen Daten
- z.B. für statistische Transaktionen
  - ungefährer Überblick, Trendanalyse
- Keine Sperren auf Tabellenebene nötig, keine Lesesperren
  - ⇨ effizient ausführbar
  - ⇨ keine anderen Transaktionen werden behindert

READ UNCOMMITTED			
Befehl	Sperre auf Datensatz	Sperre auf Tabelle	Sperrdauer
SELECT	-	-	-
UPDATE	wl	-	bis COMMIT
DELETE	wl	-	bis COMMIT
INSERT	wl	-	bis COMMIT
CREATE INDEX	-	wl	bis COMMIT
ALTER TABLE	-	wl	bis COMMIT

Isolationsebenen in SQL

READ COMMITTED

T_1	T_2
read(A)	
	write(A)
	write(B)
	commit
read(B)	
read(A)	
. . .	

- Nur Lesen endgültig geschriebener Daten
- **Non-Repeatable Read** und **Phantom-Problem** möglich
- Realisierungsmöglichkeiten:
  - Lesesperre wird unmittelbar nach Lesevorgang zurück gegeben
  - Letzter gültiger Zustand des Datenwertes wird für Leseoperation aufbewahrt (Multiversion concurrency control (MVCC))
  - Zeitstempelverfahren

READ COMMITTED			
Befehl	Sperre auf Datensatz	Sperre auf Tabelle	Sperrdauer
SELECT	rl	-	nur SELECT
UPDATE	wl	-	bis COMMIT
DELETE	wl	-	bis COMMIT
INSERT	wl	-	bis COMMIT
CREATE INDEX	-	wl	bis COMMIT
ALTER TABLE	-	wl	bis COMMIT

© 2026

Isolationsebenen in SQL

REPEATABLE READ

- Gelesene Daten einer Transaktion können durch andere Transaktionen nicht verändert werden
- **Phantom-Problem** möglich
- Realisierung
  - basiert auf einer Lese-Sperre, die bis zum Transaktionsende gehalten wird

REPEATABLE READ			
Befehl	Sperre auf Datensatz	Sperre auf Tabelle	Sperrdauer
SELECT	rl	-	bis COMMIT
UPDATE	wl	-	bis COMMIT
DELETE	wl	-	bis COMMIT
INSERT	wl	-	bis COMMIT
CREATE INDEX	-	wl	bis COMMIT
ALTER TABLE	-	wl	bis COMMIT

© 2026

Isolationsebenen in SQL
SERIALIZABLE

- Garantiert Serialisierbarkeit einer Transaktion
- Transaktion sieht nur Änderungen, die zu Beginn der Transaktion endgültig geschrieben wurden (inkl. eigene Änderungen)
- Realisierung basiert im einfachsten Fall auf einer Lese-Sperre der gesamten Tabelle (Optimierungen mittels RANGE LOCKS möglich)

SERIALIZEABLE			
Befehl	Sperre auf Datensatz	Sperre auf Tabelle	Sperrdauer
SELECT	-	rl	bis COMMIT
UPDATE	wl	-	bis COMMIT
DELETE	wl	-	bis COMMIT
INSERT	wl	-	bis COMMIT
CREATE INDEX	-	wl	bis COMMIT
ALTER TABLE	-	wl	bis COMMIT

Isolationsebenen in SQL

Mögliche Inkonsistenzen

- SQL-Standard Isolationebene bestimmen die **zulässigen Inkonsistenzen**.
- Die **Abschwächung** der Isolationsebene
  - hat den Vorteil, dass die **Parallelität** erhöht wird.
  - hat den Nachteil, dass gewisse **Inkonsistenzen** auftreten können, bzw. in Kauf genommen werden.

Isolationsebene	Dirty Read	Unrepeatable Read	Phantom Read
0 READ UNCOMMITTED	möglich	möglich	möglich
1 READ COMMITTED	-	möglich	möglich
2 REPEATABLE READ	-	-	möglich
3 SERIALIZABLE	-	-	-

- **Klassische Transaktionskonzept** entspricht **Isolationsebene 3**
- **ORACLE** (unterstützt Ebene 1 und 3)
  - garantiert Isolationsebene `1 READ COMMITTED` automatisch (mittels MVCC),
  - Isolationsebene >= 2 durch explizite Anforderung

Isolationsebenen in SQL

Beispiel - READ COMMITTED

- Transaktionen lesen nur festgeschriebene Werte.
- Allerdings können die Transaktionen unterschiedliche Zustände der Datenbasis-Objekte zu sehen bekommen.

T_1	T_2

`SET TRANSACTION`
`  ISOLATION LEVEL`
`  READ COMMITTED`
 	
`SELECT a FROM r`
`WHERE nr = 4711`
`-> alter Wert`
 	
`SELECT a FROM r`
`WHERE nr = 4711`
`-> alter Wert`
 	
`SELECT a FROM r`
`WHERE nr = 4711`
`-> neuer Wert`
	
	`UPDATE r SET a = neu`
	`WHERE nr = 4711`
	
	`COMMIT`

Isolationsebenen in SQL

Beispiel - READ COMMITTED

- Transaktionen können durch Schreibsperren blockiert werden.

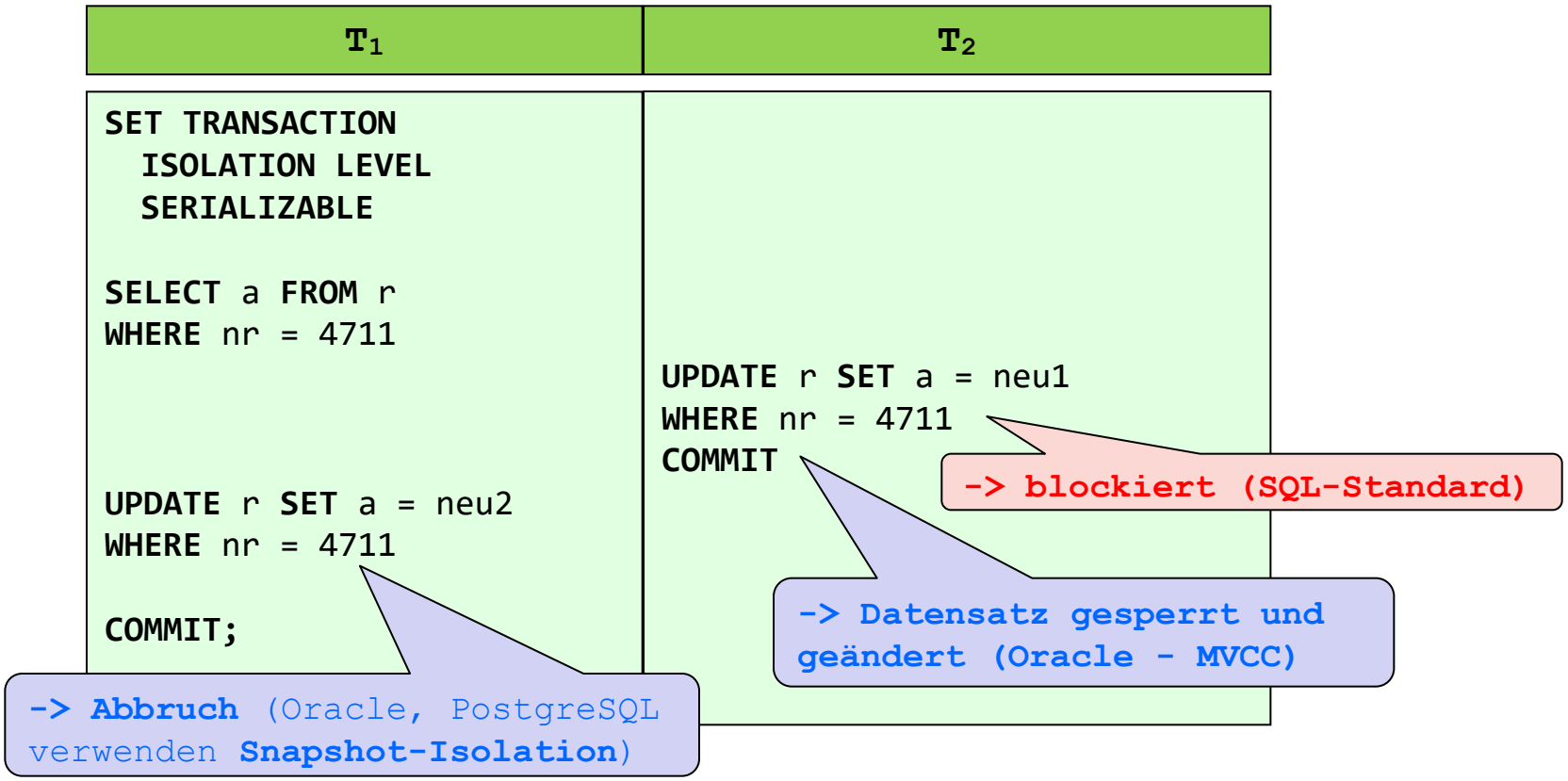
T_1	T_2

`SET TRANSACTION`
`  ISOLATION LEVEL`
`  READ COMMITTED`
 	
`SELECT a FROM r`
`WHERE nr = 4711`
`-> alter Wert`
 	
`UPDATE r SET a = neu2`
`WHERE nr = 4711`
`-> blockiert`
 	
`COMMIT`
	
	`UPDATE r SET a = neu`
	`WHERE nr = 4711`
	
	`COMMIT`

Isolationsebenen in SQL

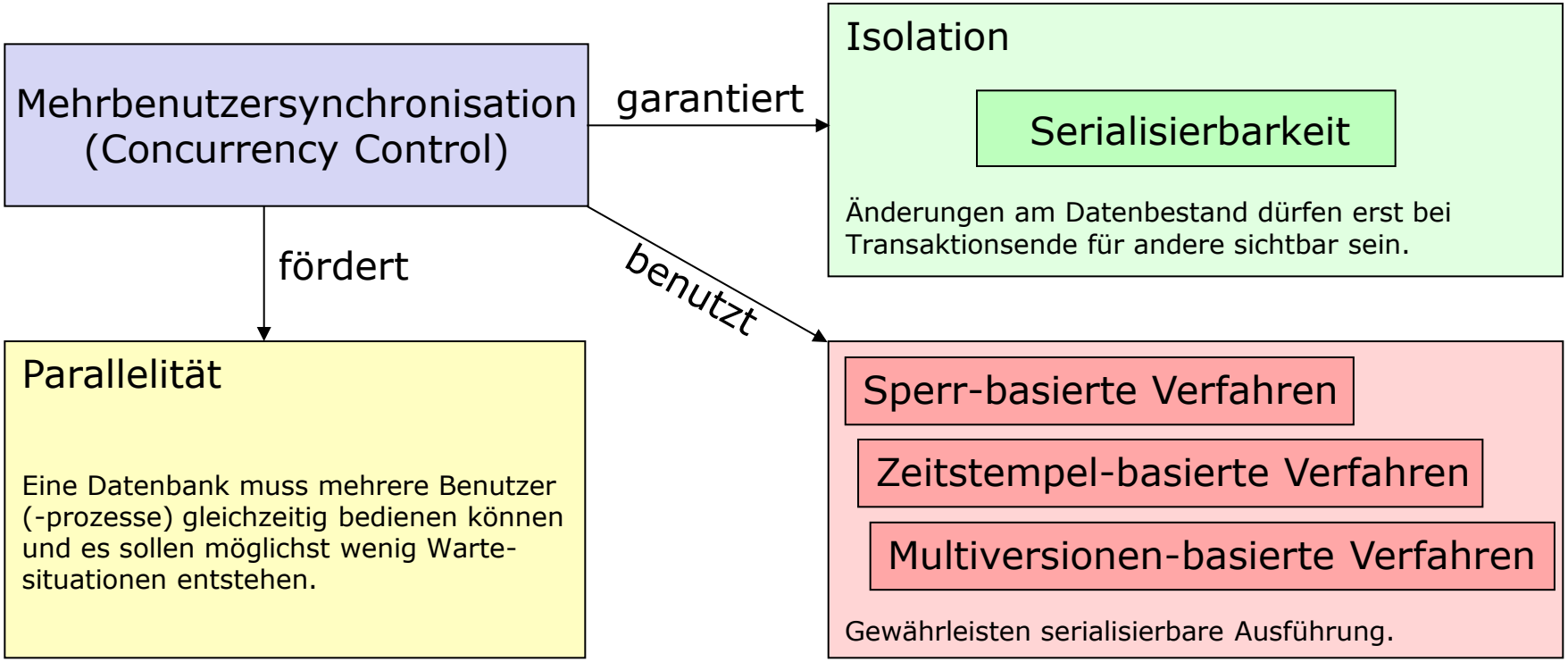
Beispiel - SERIALIZABLE

- Ablauf ist nicht serialisierbar – Transaktion wird abgebrochen.



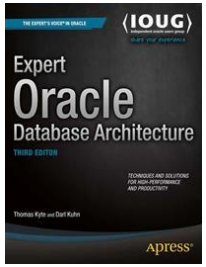
Snapshot Isolation is weaker than Serialization (= perfect serialization)!

Zusammenfassung



Anhang I

- Transaktionen in ORACLE
  - Transaktionsgrenzen
  - Sperren - Granularität und Modus
  - Multi-Version Consistency Model
  - Isolationsebenen



Der vorliegende Anhang I basiert auf:
Kyte, T., Kuhn, D.: Expert Oracle Database Architecture, Apress, 3rd Edition (2014)
(Download: https://link.springer.com/book/10.1007%2F978-1-4302-6299-2)

ORACLE
Transaktionen

- Oracle unterstützt
  - `READ COMMITTED` (Versionenverfahren - MVCC)
  - `SERIALIZABLE` mit Snapshot Isolation
  - `READ ONLY`-Modus mit Snapshot Isolation
  - kein `READ UNCOMITTED`, da MVCC eingesetzt wird
  - kein `REPEATABLE READ`, da MVCC eingesetzt wird
  - hierarchische Sperren auf Tabellen- und Datensatzebene mit intensionalen Sperren
  - keinen Lock Escalation-Mechanismus
  - Deadlock-Erkennung mit Wait-for-Graph-Algorithmus
  - keinen `UNLOCK`-Befehl → Freigabe bei `COMMIT/ROLLBACK`, d.h. striktes Zweiphasensperrprotokoll
  - keine expliziten Datensatzsperren – Sperren auf Datensätze werden implizit bei `INSERT`, `UPDATE`, `DELETE`, `MERGE`, `SELECT … FOR UPDATE` gesetzt
  - `IMMEDIATE` und `DEFERRABLE` Constraint-Überprüfungen
  - Autonome und verteilte Transaktionen
  - …

© 2026

ORACLE 1/11
Transaktionsgrenzen

- Transaktionsbeginn
  - Implizit mit erstem ausführbaren SQL-Statement nach einem `CONNECT` zur Datenbank bzw. nach einem Transaktionsende

- Transaktionsende
  - Explizit mit `COMMIT` oder `ROLLBACK` der gesamten Transaktion (kein Transaktionsende: `ROLLBACK TO SAVEPOINT <savepoint>;`)
  - Implizit mit
    - DDL-Statement - `CREATE, DROP, RENAME, ALTER`
    - `DISCONNECT` von Oracle - `COMMIT` der aktuellen Transaktion
    - abnormale Terminierung des Prozesses - `ROLLBACK` der aktuellen Transaktion

ORACLE 2/11
Sperren

- Modus
  - **Shared Locks** (S) können von mehreren Benutzern zu einem Zeitpunkt bezüglich einer Ressource gehalten werden
  - **Exclusive Locks** (X) können nur von einem Benutzer zu einem Zeitpunkt bezüglich einer Ressource gehalten werden
- Kategorie
  - Daten-Sperren (DML-Sperren): explizit u. implizit
  - Dictionary-Sperren (DDL-Sperren): implizit
- Granularität
  - gesamte Tabelle
  - einzelne Datensätze
- Status
  - erfolgreiches Reservieren der Ressource od.
  - `WAIT` bis Ressource frei ist
  - `NOWAIT`

© 2026

ORACLE 3/11
Sperren – Granularität und Modus (1)

- **Multiple Granularity Locking** zur Performanz-Steigerung

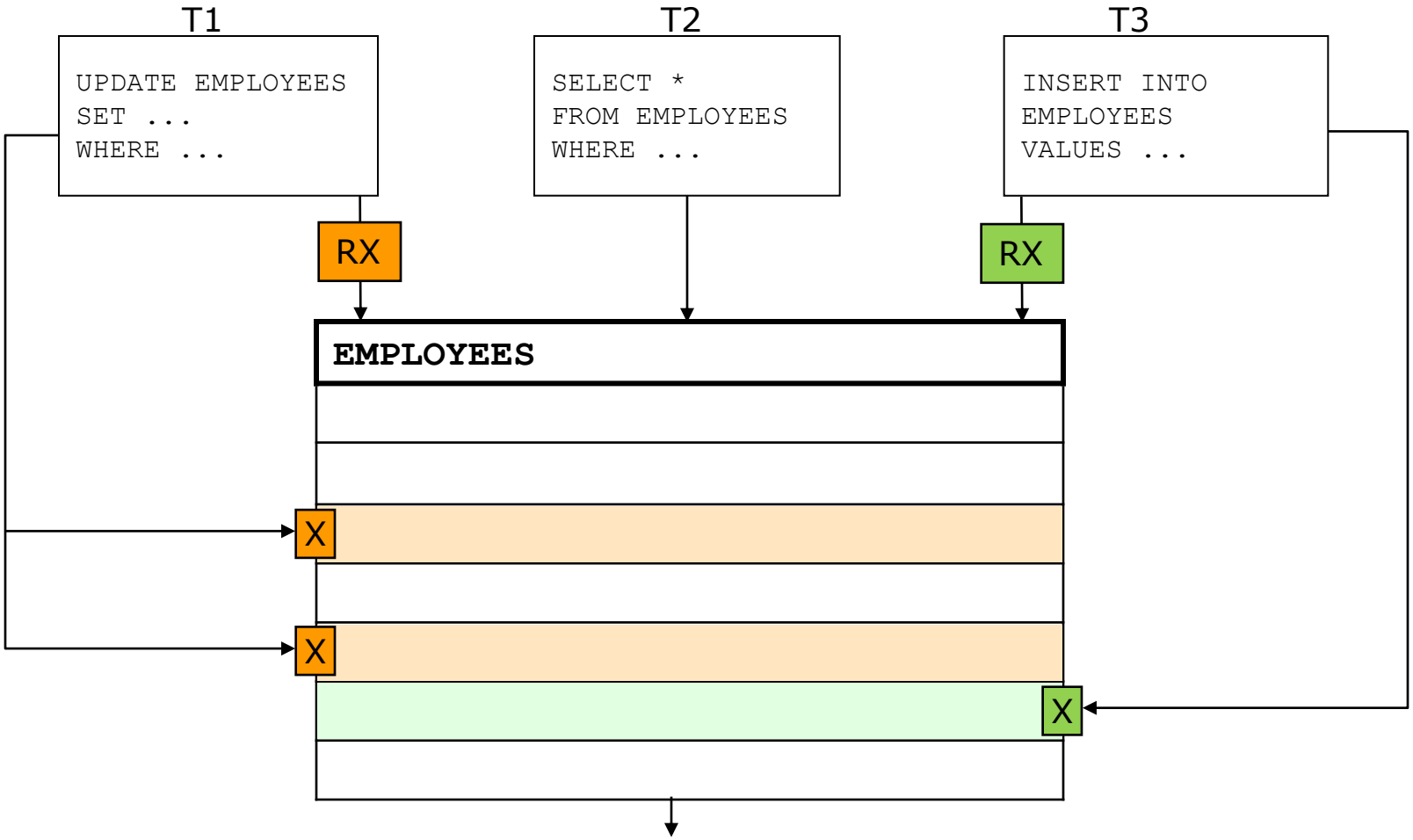
- Sperren auf Tabellen (implizit u. explizit):
  - "Herkömmliche" Sperren:
    - **S** (Share):Lesesperre
    - **X** (Exclusive): Schreibsperre
  - "Intensionale" Sperren:
    - **RS** (Row Share): Intensionale Lesesperre
    - **RX** (Row Exclusive): Intensionale Schreibsperre
    - **SRX** (Share Row Exclusive): Lesesperre u. Intensionale Schreibsperre
  - Kein Lock Escalation

- Sperren auf Datensätze (nur implizit):
  - X (Exclusive): Schreibsperre

© 2026

ORACLE 4/11

Sperren – Granularität und Modus (2)



ORACLE 5/11
Sperren – Granularität und Modus (3)

Implizite Sperren durch SQL-Statements	Sperre auf Datensatz	Sperre auf Tabelle
SELECT ... FROM table ...	-	-
INSERT INTO table ...	X	RX
UPDATE table ...	X	RX
DELETE FROM table ...	X	RX
SELECT ... FROM table ... **FOR UPDATE** OF ...	X	RX
DDL Statement	-	X

Explizite Sperren	Sperre auf Datensatz	Sperre auf Tabelle
LOCK TABLE table IN ... [NOWAIT]		
ROW SHARE MODE	-	RS
ROW EXCLUSIVE MODE	-	RX
SHARE MODE	-	S
SHARE ROW EXCLUSIVE MODE	-	SRX
EXCLUSIVE MODE	-	X

© 2026

ORACLE 6/11
Kompatibilität der Daten-Locks

SQL Statement	Tabellen-Lock	Erlaubte Lock Modi				
		RS	RX	S	SRX	X
SELECT ... FROM ... table	none	✓	✓	✓	✓	✓
INSERT INTO table ...	RX	✓	✓	✗	✗	✗
UPDATE table ...	RX	✓*	✓*	✗	✗	✗
DELETE FROM table ...	RX	✓*	✓*	✗	✗	✗
SELECT ... FROM table **FOR UPDATE** OF ...	RX	✓*	✓*	✗	✗	✗
LOCK TABLE table IN ... [NOWAIT]						
ROW SHARE MODE	RS	✓	✓	✓	✓	✗
ROW EXCLUSIVE MODE	RX	✓	✓	✗	✗	✗
SHARE MODE	S	✓	✗	✓	✗	✗
SHARE ROW EXCLUSIVE MODE	SRX	✓	✗	✗	✗	✗
EXCLUSIVE MODE	X	✗	✗	✗	✗	✗

*... nur "✓" falls von anderer T nicht die gleichen Datensätze gesperrt werden

© 2026

ORACLE 7/11
Multi-Version Consistency Model (1)

- Abfragen fordern keine Sperren an - **Non-Blocking Queries**
  - Andere Transaktionen können eine Tabelle auf der eine Abfrage Q1 arbeitet abfragen od. aktualisieren - inklusive der von Q1 abgefragten Datensätze
  - Abfragen brauchen nicht auf die Freigabe von Datensperren zu warten - sie können immer weiterarbeiten
- Jedem Benutzer wird eine **Read-Consistent View** zur Verfügung gestellt
- Diese Read-Consistent View erstreckt sich entweder
  - auf die Dauer eines einzelnen Statements: Statement-Level Read Consistency (standardmäßig) oder
  - auf die Dauer einer ganzen Transaktion: Transaction-Level Read Consistency (nur im Isolation Level Serializable)

ORACLE 8/11
Multi-Version Consistency Model (2)

- Statement-Level Read Consistency
  - garantiert, dass die Ergebnisdaten eines einzelnen `SELECT` bezüglich des `SELECT`-Beginnzeitpunktes konsistent sind
  - sieht Änderungen vorangegangener DML-Statements (ausgenommen bei Unterabfragen) in der gleichen Transaktion
  - sieht nicht Änderungen
    - anderer noch aktiver Transaktionen oder
    - jener Transaktionen, die während des `SELECT` ein `COMMIT` durchführen

© 2026

ORACLE 9/11
Multi-Version Consistency Model (3)

- Transaction-Level Read Consistency
  - Garantiert, dass die Ergebnisdaten mehrerer `SELECT` innerhalb einer Transaktion bezüglich des Transaktionsbeginnzeitpunktes konsistent sind (Repeatable Reads)
  - 3 alternative Mechanismen:
    - `SET TRANSACTION READ ONLY`
      - » muss die erste Anweisung einer Transaktion sein
      - » konsistentes Lesen über Zugriff auf Rollback-Segment
      - » keine DML-Statements in Read-Only-Transaktion erlaubt, in anderen Transaktionen schon
    - Explizites Locking: S, SRX oder X-Lock:
      - » verhindern DML-Statements anderer Transaktionens von vornherein (nur SELECT), da kein RX-Lock gewährt wird
      - » v. a. sinnvoll, wenn eine Transaktionen Daten liest u. in Abhängigkeit von den gelesenen Werten Änderungen durchführt !
    - Änderungen am Locking-Verhalten (Isolation Level: Serializable)

© 2026

ORACLE 10/11
Isolationsebenen

Isolationsebenen	
`READ COMMITTED`	garantiert Statement-Level Read Consistency (default) ⇨ verhindert NICHT Non-Repeatable-Reads, Phantome
`SERIALIZABLE`	SELECT fordert Share Lock an ⇨ garantiert Transaction-Level Read Consistency "ORA-08177: Cannot serialize access for this transaction" kann auftreten

- Einstellbar durch:
  - `SET TRANSACTION ISOLATION LEVEL <level>;`
    muss die erste Anweisung einer Transaktion sein
  - `ALTER SESSION SET ISOLATION_LEVEL = <level>;`
    gilt für alle nachfolgenden Transaktionen der Session

ORACLE 11/11
Isolationsebenen – SQL-Standard versus Oracle

- Read Locking versus Oracle's Multiversioning

Isolation Level	Implementation	Writes Block Reads	Reads Block Writes	Deadlock-Sensitive Reads	Incorrect Query Results	Lost Updates	Lock Escalation
READ UNCOMITTED	Not Oracle	No	No	No	Yes	Yes	Yes
READ COMMITTED	Not Oracle	Yes	No	No	Yes	Yes	Yes
READ COMMITTED	**Oracle**	No	No	No	No	No*	No
REPEATABLE READ	Not Oracle	Yes	Yes	Yes	No	No	Yes
SERIALIZABLE	Not Oracle	Yes	Yes	Yes	No	No	Yes
SERIALIZABLE	**Oracle**	No	No	No	No	No	No

* mit SELECT FOR UPDATE NOWAIT

Kyte, T., Kuhn, D.: Expert Oracle Database Architecture, Apress, 3rd Edition (2014), p 274

Anhang II

- Lange Transaktionen
  - Kurze versus lange Transaktionen
  - Checkout/Checkin-Verfahren
  - Zeitstempel/Prüfregel-Verfahren

© 2026

Kurze und lange Transaktionen 1/2

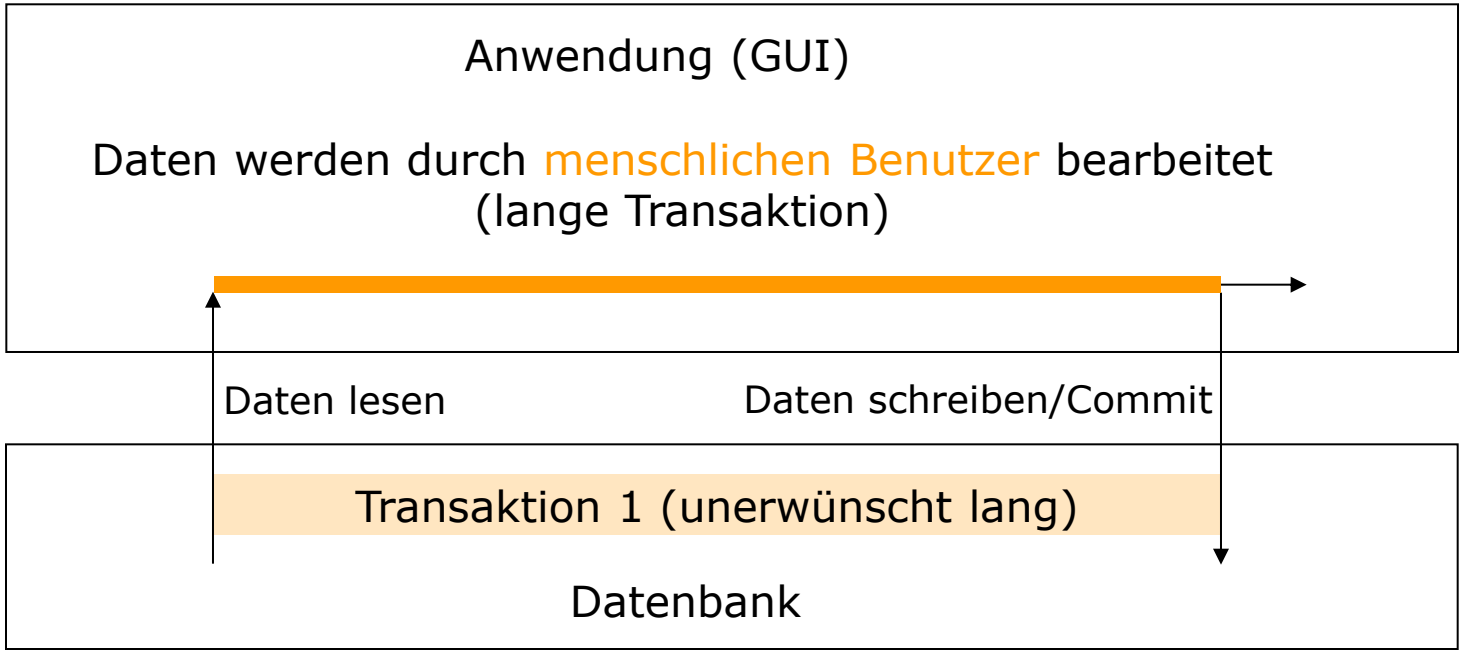
- Bisher: **Kurze, technische Transaktionen**
  - Oberstes Ziel ist ein serialisierbarer Ablauf
  - Keine spezifische Semantik bei Konflikten, sondern Abbruch der Transaktion
  - Keine Benutzerinteraktion während der Transaktion (Alle Informationen und Regeln zur Durchführung der Transaktion sind vor der Transaktionsausführung bekannt)
  - Minimalen Zeitbedarf anstreben

© 2026

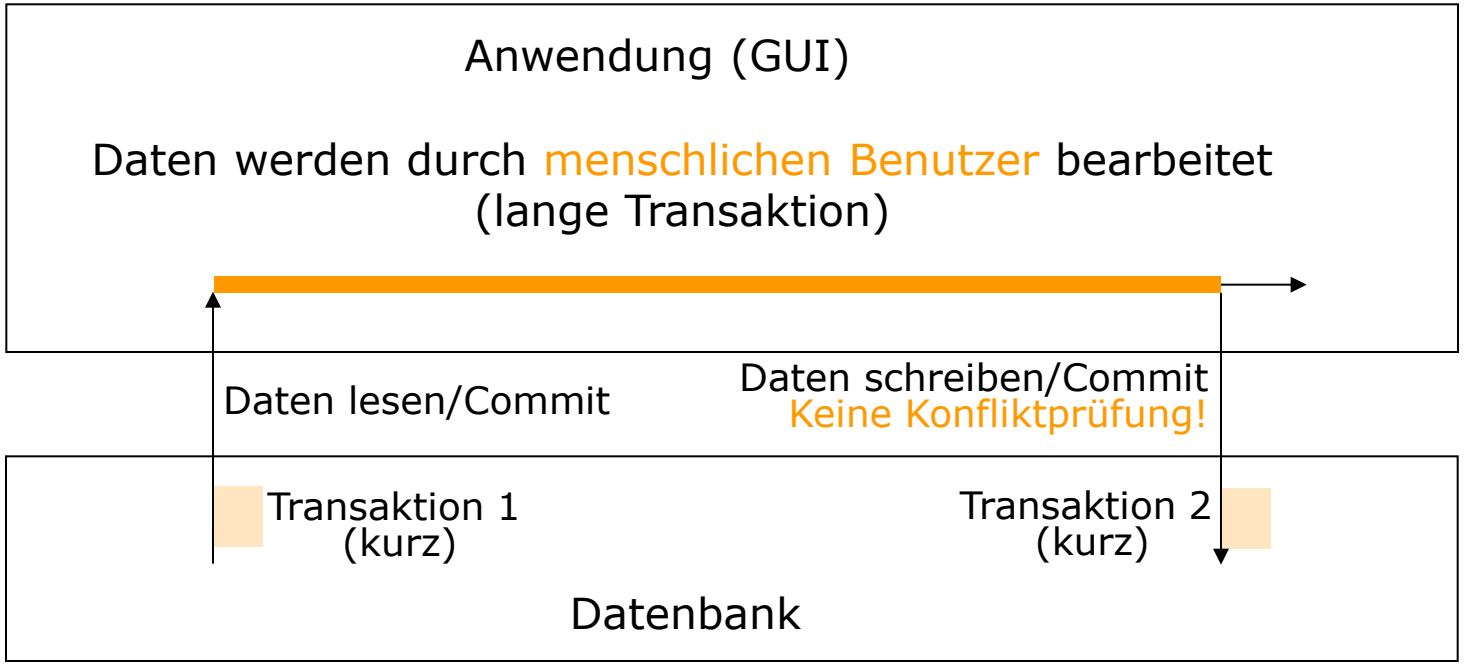
Kurze und lange Transaktionen 2/2

- Neu: **Lange, logische Transaktionen**
  - Daten aus der Datenbank entnehmen für Anwendungen
  - Bearbeitung in der Anwendung beliebig lange
  - Benutzerinteraktion während der Transaktion muss möglich sein, da nicht alle Informationen oder Regeln explizit bekannt sind
  - Zurückschreiben in die Datenbank
  - Basis für die Entnahme und Zurückschreiben: Kurze Transaktionen

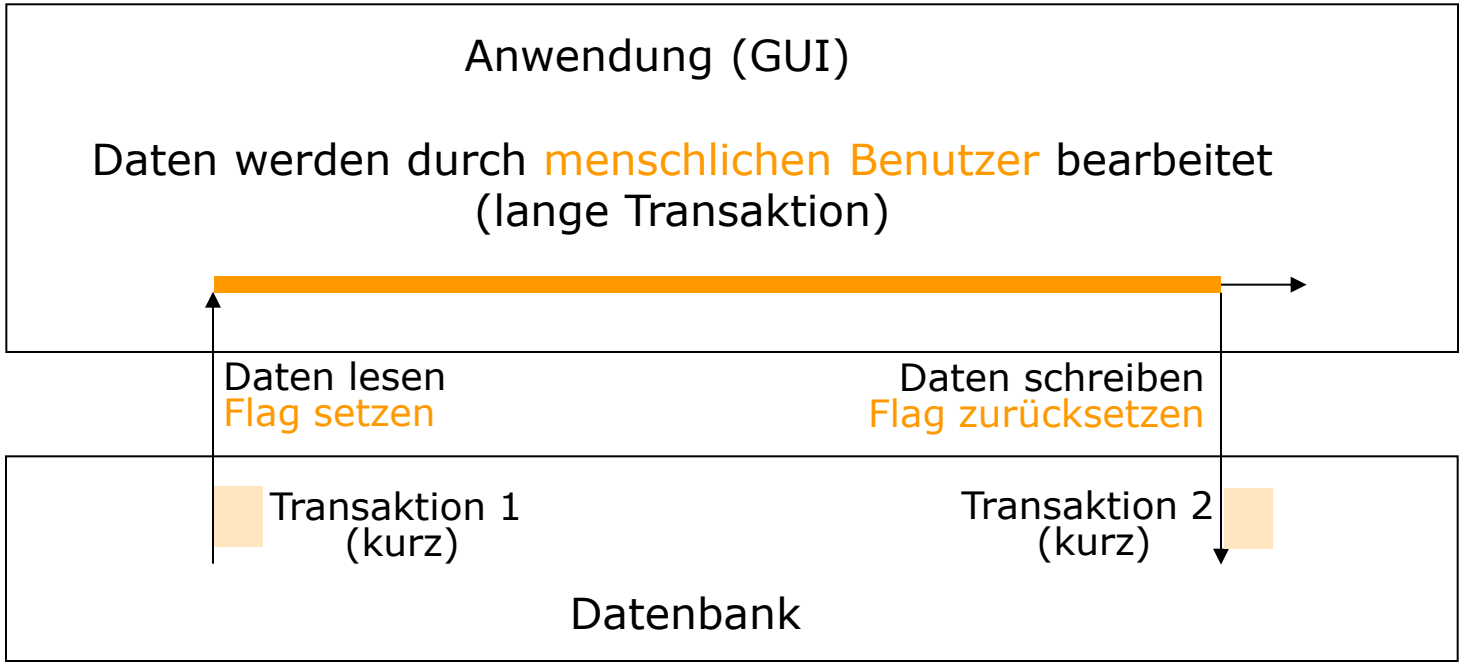
Problematisches Vorgehen 1/2



Problematisches Vorgehen 2/2



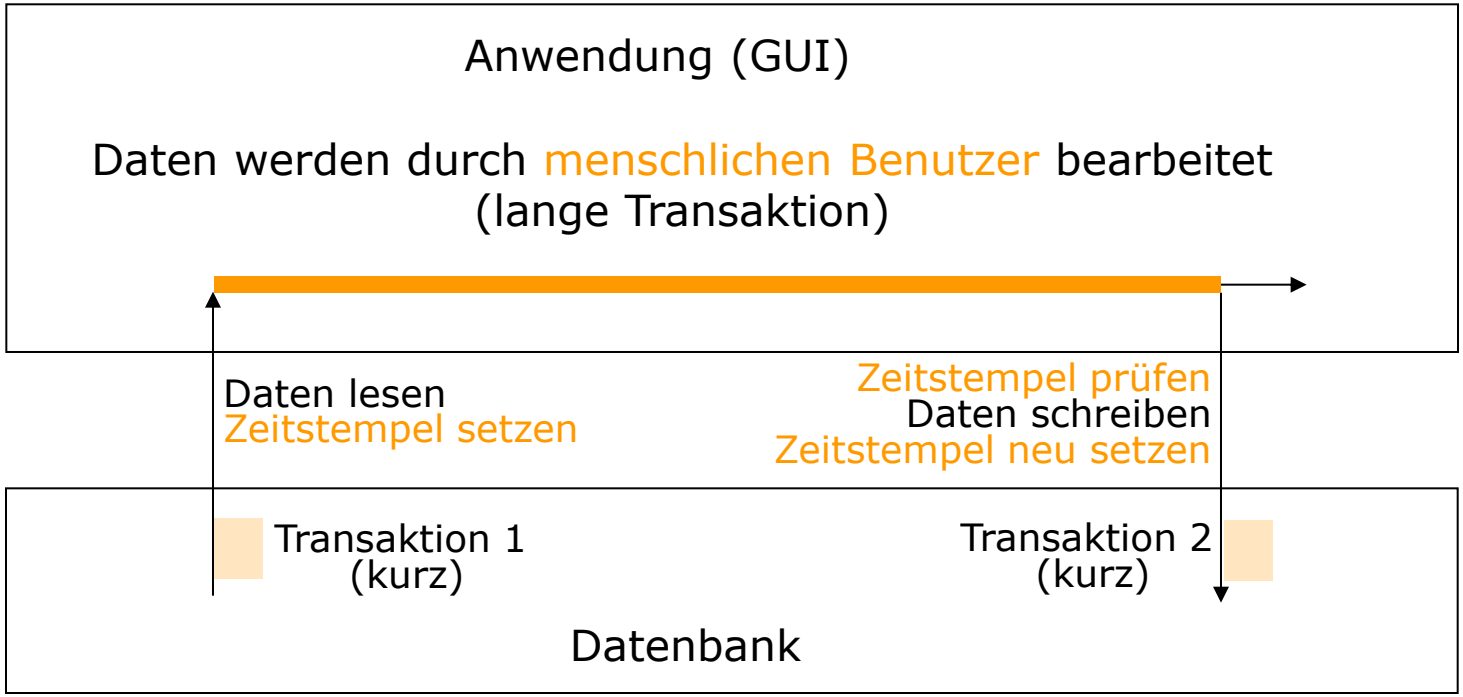
Lösung 1/2
Checkout/Checkin



Flag: ModifiedDate (Timestamp), VersionNumber (Hash Value), BinaryFlag+SessionId, …

Flag-Verwaltung: Anpassen Datenbankschema, Verwaltung durch Framework/OR-Mapper, …

Lösung 2/2
Zeitstempel/Prüfregel



Transaktion versus Verbindung

- Eine **kurze Transaktion** wird immer über **eine Verbindung** abgewickelt.
- Über **eine Verbindung** können **mehrere Transaktionen** abgewickelt werden, aber immer nur *nacheinander*.
- Kurze Transaktionen können die zugrunde liegende Verbindung nicht überdauern.
- Ein **Verbindungsabbruch** hat immer einen **Transaktionsabbruch** zur Folge.
- Lange Transaktionen enthalten unter Umständen eine gewollte oder ungewollte Unterbrechung der Verbindung: Sie müssen in der Regel selber implementiert werden.

Unterstützung für lange Transaktionen

- Das Zeitstempelverfahren erlaubt das Abwickeln langer Transaktionen (`READ COMMITTED`-Garantie und Vermeidung von Lost Updates) innerhalb *einer* technischen Transaktion/Verbindung.
- Das Versionenverfahren von Oracle erlaubt das Abwickeln langer Lesetransaktionen (Daten mit `REPEATABLE READ` Garantie) innerhalb *einer* technischen Transaktion/Verbindung.

Anhang III: Definitionen

Begriff	Informale Bedeutung
Transaktion	Eine Transaktion ist eine Folge von Operationen (Aktionen), die die Datenbank von einem konsistenten Zustand in einen konsistenten, eventuell veränderten, Zustand überführt, wobei das ACID-Prinzip eingehalten werden muss.
Atomarität	Transaktion wird entweder ganz oder gar nicht ausgeführt.
Konsistenz	Datenbank ist vor Beginn und nach Beendigung einer Transaktion jeweils in einem konsistenten Zustand.
Isolation	Nutzer, der mit einer Datenbank arbeitet, sollte den Eindruck haben, dass er mit dieser Datenbank alleine arbeitet.
Dauerhaftigkeit	nach erfolgreichem Abschluss einer Transaktion muss das Ergebnis dieser Transaktion „dauerhaft" in der Datenbank gespeichert werden.
Serialisierbarkeit	Eine verschränkte Ausführung mehrerer Transaktionen heißt serialisierbar, wenn ihr Effekt identisch zum Effekt einer (beliebig gewählten) seriellen Ausführung dieser Transaktionen ist.
COMMIT	Mit dem COMMIT-Befehl wird unter SQL eine Transaktion bestätigt und die Änderungen werden in die Datenbank übernommen.

© 2026